RAJALAKSHMI ENGINEERING COLLEGE
RAJALAKSHMI NAGAR, THANDALAM – 602 105

RAJALAKSHMI ENGINEERING COLLEGE

CS23334 – Fundamentals of Data Science Lab

Laboratory Record Note Book

Name :	PRADEEPA M
Year / Branch / Section :	2nd year / CSE / B
University Register No. :	2116250701530
College Roll No. :	250701530
Semester :	III
Academic Year :	2026-2027

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

CS23334 – Fundamentals of Data Science Lab

NAME	PRADEEPA M
ROLL NO	250701530
YEAR	2nd year
SEC	B

RAJALAKSHMI ENGINEERING COLLEGE
An Autonomous Institution

BONAFIDE CERTIFICATE

Name: PRADEEPA M

Academic Year: 2026-2027 **Semester:** III **Branch:** CSE

Register No. 211625070153O

Certified that this is the bonafide record of work done by the above student in the Fundamentals of Data Science *Laboratory during the academic year 2026 – 2027.*

Signature of Faculty in-charge

Submitted for the Practical Examination held on .....

Internal Examiner External Examiner

INDEX

Sl.NO	Experiment	Date	GitHub
1.a	Analyze the trend of data science job postings over the last decade for the given data on the number of data science job postings each year. Use pandas for data manipulation and matplotlib /seaborn for visualization.	29/07/26	
1.b	Analyze and visualize the distribution of various data science roles (Data Analyst, Data Engineer, Data Scientist, etc.) from a dataset.	29/07/26	
1.c	Conduct an experiment to differentiate Structured , Un-structured and Semi structured data based on data sets given.	05/08/26	
1.d	Conduct an experiment to encrypt and decrypt given sensitive data Use the cryptography library fernet.	05/08/26	
2	Upload and Analyze the data set given in csv format and perform data preprocessing and visualization.	12/08/26	
3.a	Conduct an experiment to show data visualization using line plot	19/08/26	
3.b	Conduct an experiment to show data visualization using bar chart	19/08/26	
3.c	Conduct an experiment to show data visualization using pie chart	19/08/26	
4	To Count the frequency of occurrence of a word in a body of text is often needed during text processing.	26/08/26	
5	Data Collection and Initial Exploration	16/09/26	
6	Time Series Analysis of Website Traffic	16/09/26	

7	Random Sampling and Sampling Distribution	23/09/26	
8	Conduct Z test for the Data given using Python	23/09/26	
9	Experiment to understand Matplotlib library use cases in Data Science through visualization.	23/09/26	
10	Experiment to understand array function in Data science.	23/09/26	
11	Experiment to understand pandas library use cases in Data science.	30/09/26	
12	Experiment to detect outliers in a given data set.	30/09/26	
13	Experiment to handle missing values and inappropriate data in a given data set.	30/09/26	
14	Experiment to understand feature scaling	30/09/26	
15	Experiment to understand the data preprocessing in Data science	30/09/26	
16	Experiment to understand EDA-Quantitative and Qualitative analysis.	07/10/26	
17	Experiment to understand Linear Regression for a given data set.	07/10/26	
18	Experiment to understand KNN algorithm for a given data set.	07/10/26	
19	Experiment to understand Logistic Regression for a given data set.	07/10/26	
20	Experiment to understand K-means clustering algorithm for a given data set.	07/10/26	

EXPERIMENT NO: 1.(a)

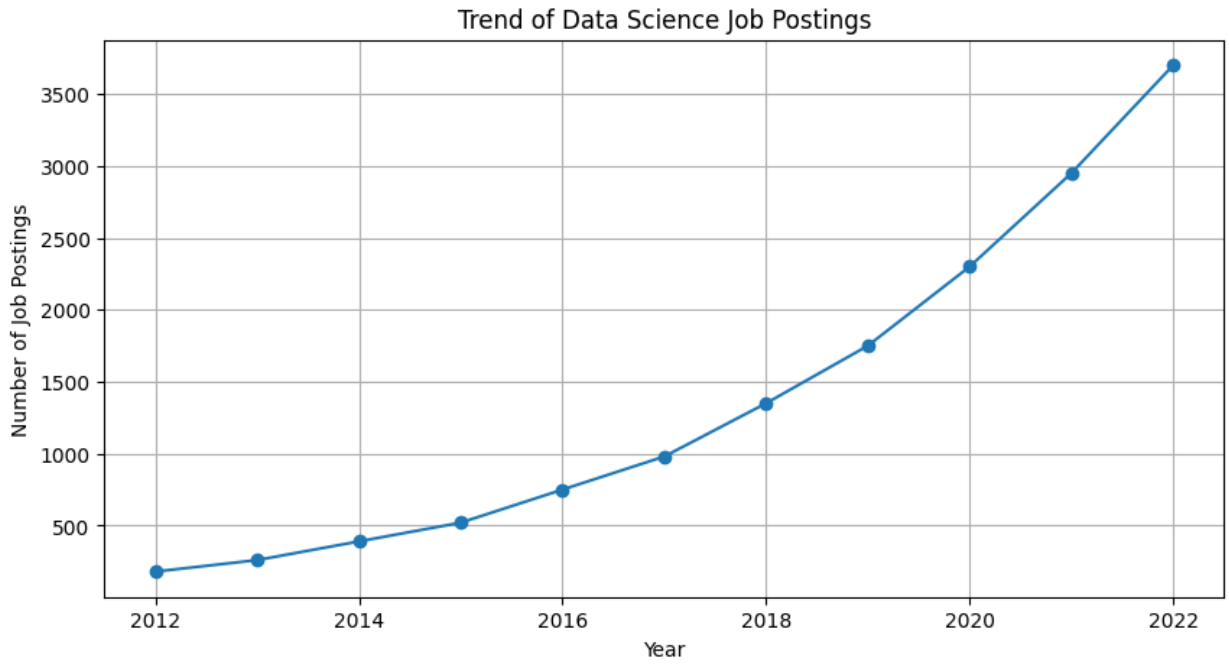
Aim:

To analyse the trend of data science job postings over a decade using Pandas and Matplotlib.

Code:

```
import pandas as pd
import matplotlib.pyplot as plt
data = {
    'Year': list(range(2012, 2023)),
    'Job Postings': [180, 260, 390, 520, 750, 980, 1350, 1750, 2300, 2950, 3700]
}
df = pd.DataFrame(data)
print(df)
plt.figure(figsize=(10, 5))
plt.plot(df['Year'], df['Job Postings'], marker='o')
plt.title('Trend of Data Science Job Postings')
plt.xlabel('Year')
plt.ylabel('Number of Job Postings')
plt.grid(True)
plt.show()
```

OUTPUT:



RESULT:

The line graph shows an increasing trend on data science job postings from 2012 to 2022. The number of postings increased from 180 in 2012 to 3700 in 2022, indicating a significant growth in demand for data science professionals.

EXPERIMENT NO: 1.(b)

Aim:

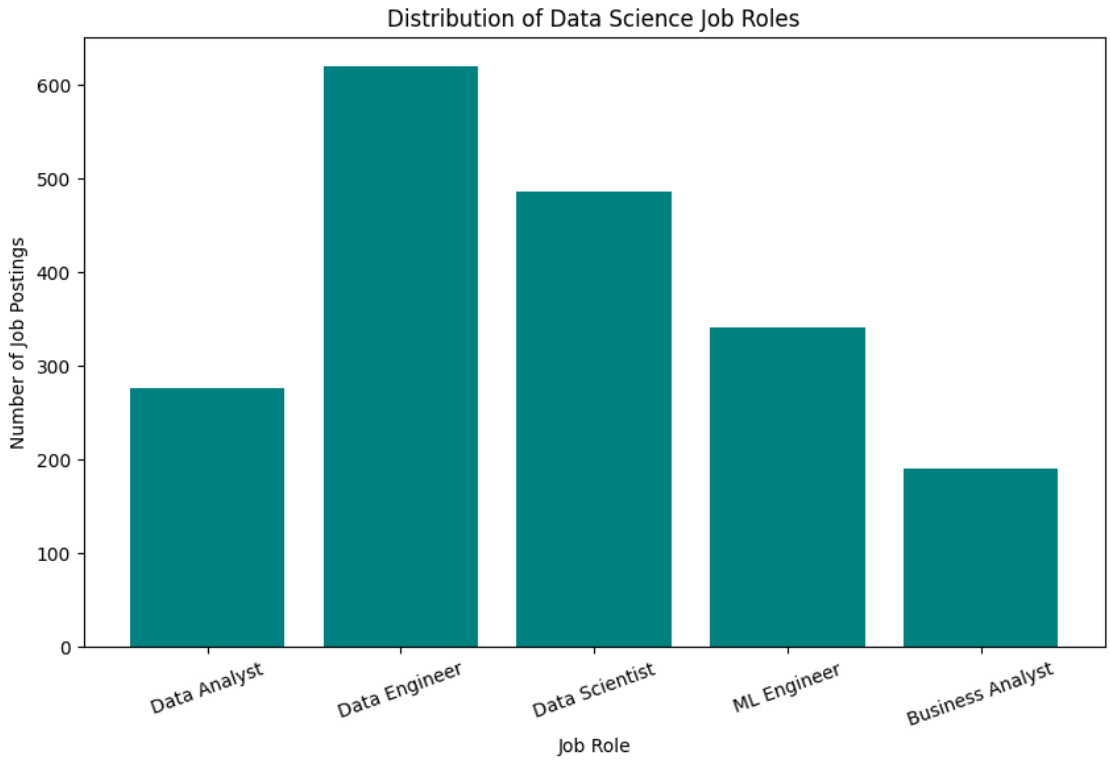
To analyse and visualize the distribution of different data science job roles using Pandas and Matplotlib.

Code:

```
import pandas as pd
import matplotlib.pyplot as plt
data = {
    'Role': [
        'Data Analyst',
        'Data Engineer',
        'Data Scientist',
        'ML Engineer',
        'Business Analyst'
    ],
    'Count': [275, 620, 485, 340, 190]
}
df = pd.DataFrame(data)
print("Data Science Job Roles Dataset:")
print(df)
plt.figure(figsize=(10, 6))
plt.bar(
    df['Role'],
    df['Count'],
    color='teal')
```

```
plt.title('Distribution of Data Science Job Roles')
plt.xlabel('Job Role')
plt.ylabel('Number of Job Postings')
plt.xticks(rotation=20)
plt.show()
```

OUTPUT:



RESULT:

The bar graph shows that Data Engineer has the highest number of Job postings with 620. Business Analyst has the lowest with 190 postings. The graph helps compare the demand for different data-science-related roles.

EXPERIMENT NO: 1.(c)

Aim:

To differentiate between structured, unstructured and semi-structured data using suitable examples.

Code:

```
import pandas as pd
structured_data = pd.DataFrame({
    'ID': [101, 102, 103],
    'Name': ['David', 'Emma', 'John'],
    'Age': [28, 32, 26]
})
print("Structured Data:\n")
print(structured_data)
unstructured_data = (
    "Machine learning is a branch of artificial intelligence. "
    "It is used to analyze data and make predictions."
)
print("\nUnstructured Data:\n")
print(unstructured_data)
semi_structured_data = {
    'ID': 201,
    'Name': 'Sophia',
    'Details': {
        'Department': 'Data Science',
        'Experience': 3
```

```
    }
}
print("\nSemi-structured Data:\n")
print(semi_structured_data)
```

OUTPUT:

```
Structured Data:
   ID   Name  Age
0  101  David   28
1  102   Emma   32
2  103   John   26
Unstructured Data:
Machine learning is a branch of artificial intelligence. It is used to analyze data and make predictions.
Semi-structured Data:
{'ID': 201, 'Name': 'Sophia', 'Details': {'Department': 'Data Science', 'Experience': 3}}
```

RESULT:

The experiment successfully differentiated the three types of data based on their structure and format.

EXPERIMENT NO: 1.(d)

Aim:

To encrypt and decrypt sensitive data using the Fernet Cryptography library in Python.

Code:

```
from cryptography.fernet import Fernet

key = Fernet.generate_key()

cipher_suite = Fernet(key)

plain_text = b"Student ID: DS2024-105"

cipher_text = cipher_suite.encrypt(plain_text)

decrypted_text = cipher_suite.decrypt(cipher_text)

print("Original Data:", plain_text)

print("Encrypted Data:", cipher_text)

print("Decrypted Data:", decrypted_text)
```

OUTPUT:

Original Data: b'Student ID: DS2024-105'

Encrypted Data:
b'gAAAAABqhIyJpVUicEqCL5Ny5uEBwGJIGI3dI_p1oBt0HxlhdQU3GhY5IPhLH_uyzix_GdWCVw-e7wUIUOcY4POfrZtpWA-Y9N23HxEh8hDn152RaW29dV0='

Decrypted Data: b'Student ID: DS2024-105'

RESULT:

The sensitive data was successfully encrypted and decrypted using the Fernet cryptography library.

EXPERIMENT 2

AIM:

To upload a CSV dataset, perform data preprocessing and statistical analysis using Pandas and visualize the sales data using Pandas and Seaborn.

CODE:

```
import pandas as pd

import numpy as np

import matplotlib.pyplot as plt

import seaborn as sns


file_path = 'sales-data-new.csv'

df = pd.read_csv(file_path)


print(df.head())


print(df.isnull().sum())


df['Sales'] = df['Sales'].fillna(df['Sales'].mean())


df.dropna(subset=['Product', 'Quantity', 'Region'], inplace=True)


print(df.describe())


product_summary = df.groupby('Product').agg({

    'Sales': 'sum',
```

```
    'Quantity': 'sum'
}).reset_index()

print(product_summary)

plt.figure(figsize=(10, 6))
plt.bar(product_summary['Product'], product_summary['Sales'])
plt.xlabel('Product')
plt.ylabel('Total Sales')
plt.title('Total Sales by Product')
plt.show()

df['Date'] = pd.to_datetime(df['Date'], dayfirst=True)

sales_over_time = df.groupby('Date').agg({
    'Sales': 'sum'
}).reset_index()

plt.figure(figsize=(10, 6))
plt.plot(sales_over_time['Date'], sales_over_time['Sales'])
plt.xlabel('Date')
plt.ylabel('Total Sales')
plt.title('Sales Over Time')
plt.xticks(rotation=45)
plt.show()
```

```
pivot_table = df.pivot_table(
    values='Sales',
    index='Region',
    columns='Product',
    aggfunc='sum',
    fill_value=0
)

print(pivot_table)

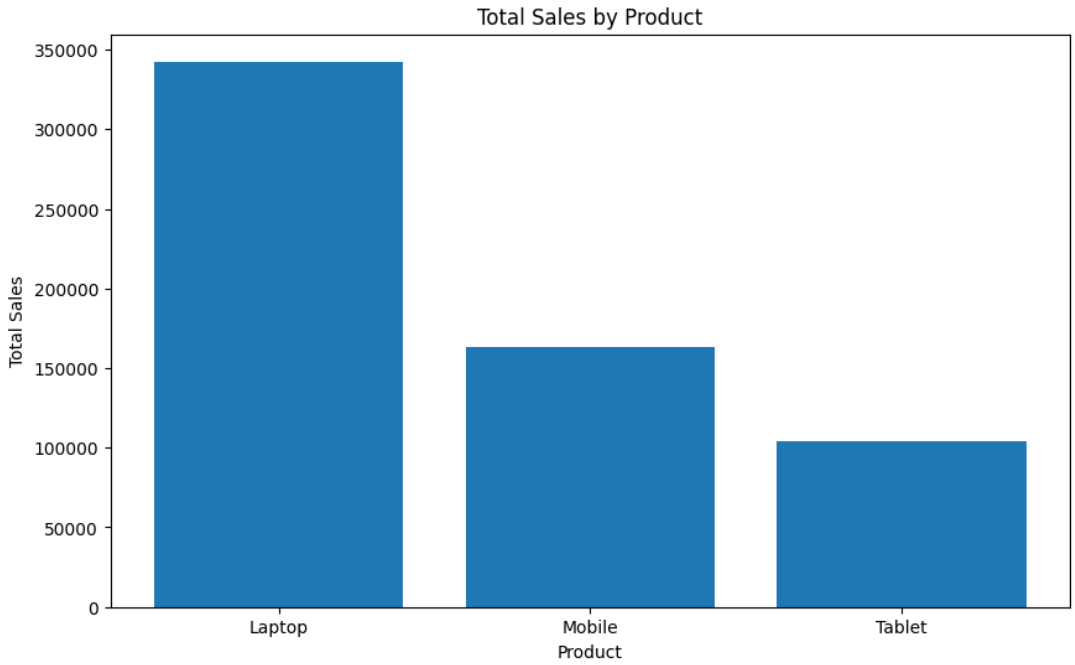
correlation_matrix = df[['Sales', 'Quantity']].corr()

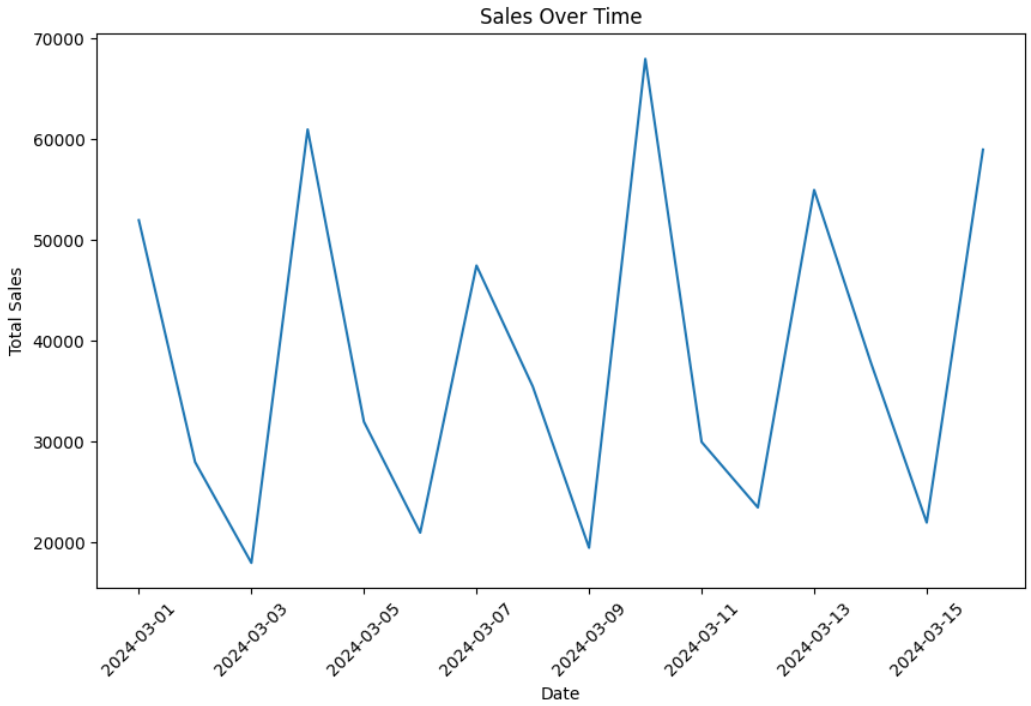
print(correlation_matrix)

plt.figure(figsize=(8, 6))
sns.heatmap(correlation_matrix, annot=True, cmap='coolwarm')
plt.title('Correlation Matrix')
plt.show()
```

OUTPUT:

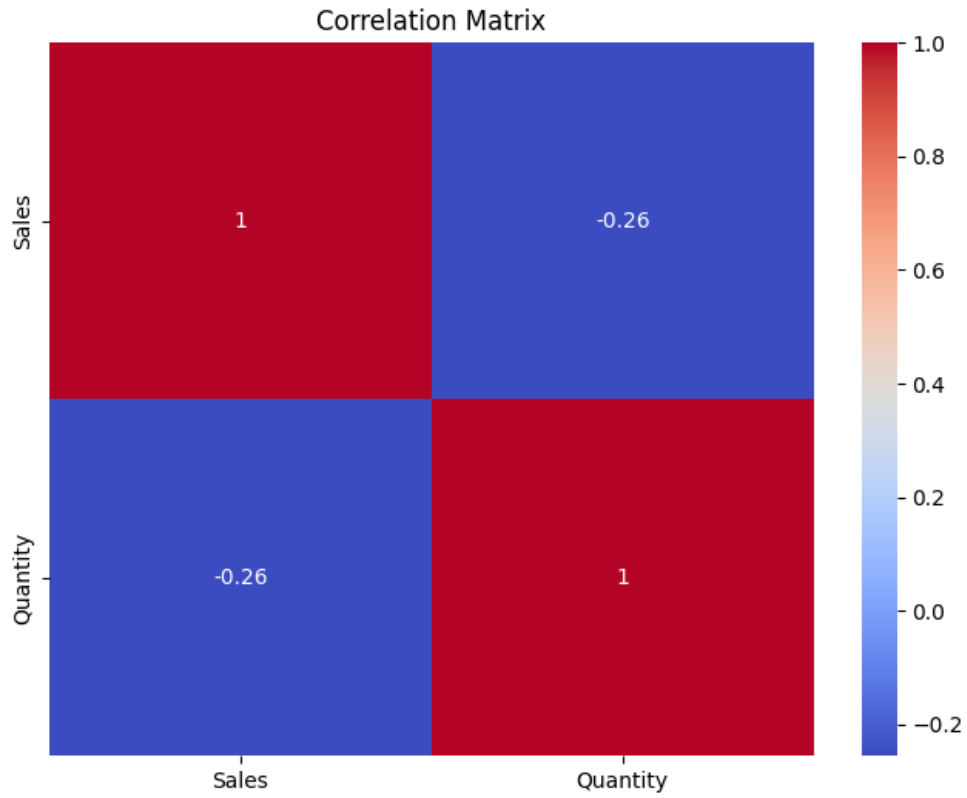
```
         Date Product  Sales  Quantity Region
0  01-03-2024  Laptop  52000         4  North
1  02-03-2024  Mobile  28000         7  South
2  03-03-2024  Tablet  18000         5   East
3  04-03-2024  Laptop  61000         5   West
4  05-03-2024  Mobile  32000         8  South
Date        0
Product     0
Sales       0
Quantity    0
Region      0
dtype: int64
              Sales   Quantity
count     16.000000  16.000000
mean   38125.000000   5.750000
std    16630.794729   1.693123
min    18000.000000   3.000000
25%    23125.000000   4.750000
50%    33750.000000   5.500000
75%    52750.000000   7.000000
max    68000.000000   9.000000
  Product  Sales  Quantity
0  Laptop 342500        27
1  Mobile 163500        39
2  Tablet 104000        26
```





Product	Laptop	Mobile	Tablet
Region			
East	0	38000	58500
North	222500	0	0
South	0	90000	22000
West	120000	35500	23500

	Sales	Quantity
Sales	1.0000	-0.2557
Quantity	-0.2557	1.0000



RESULT:

The dataset was successfully loaded and analysed using Pandas. The preprocessing operation was performed successfully and the imputation of missing values were present.

EXPERIMENT 3(a)

AIM:

To perform data visualization using a line plot to compare the sales of two products, Coke and Pepsi over different years.

CODE:

```
import matplotlib.pyplot as sales

Years = [2020, 2021, 2022, 2023, 2024]
Coke_Sales = [120, 135, 150, 165, 180]
Pepsi_Sales = [110, 125, 145, 155, 170]

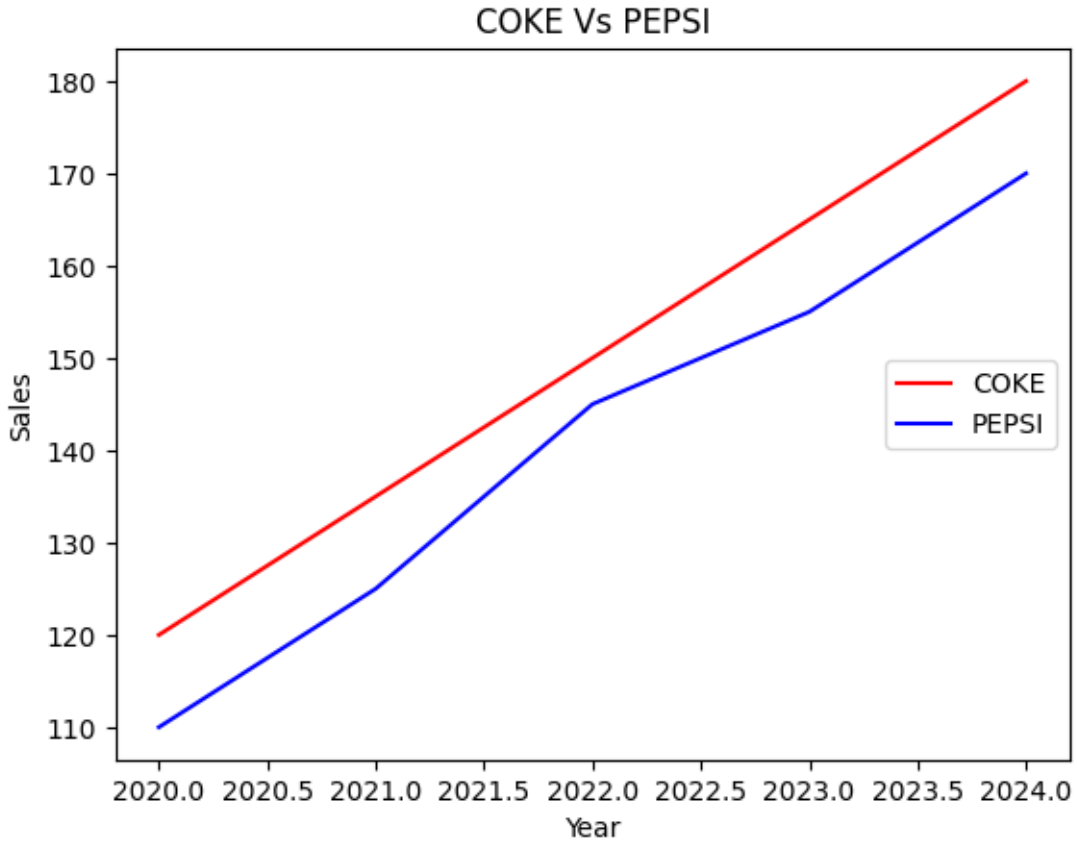
sales.title("COKE Vs PEPSI")
sales.xlabel("Year")
sales.ylabel("Sales")

sales.plot(Years, Coke_Sales, color="red", label="COKE")
sales.plot(Years, Pepsi_Sales, color="blue", label="PEPSI")

sales.legend(loc="center right")

sales.show()
```

OUTPUT:



RESULT:

The sales data of Coke and Pepsi was successfully loaded and analysed. The line plot was generated successfully, and the missing values were present.

EXPERIMENT 3(b)

AIM:

To perform data visualization using a bar chart to compare the sales of different products.

CODE:

```
import matplotlib.pyplot as sales

import numpy as np


Products = ['Laptop', 'Mobile', 'Tablet', 'Watch', 'Headphone', 'Camera']

xaxis = np.arange(len(Products))

Sales = [85, 120, 65, 95, 110, 50]


sales.bar(Products, Sales)

sales.xticks(xaxis, Products, rotation=45)

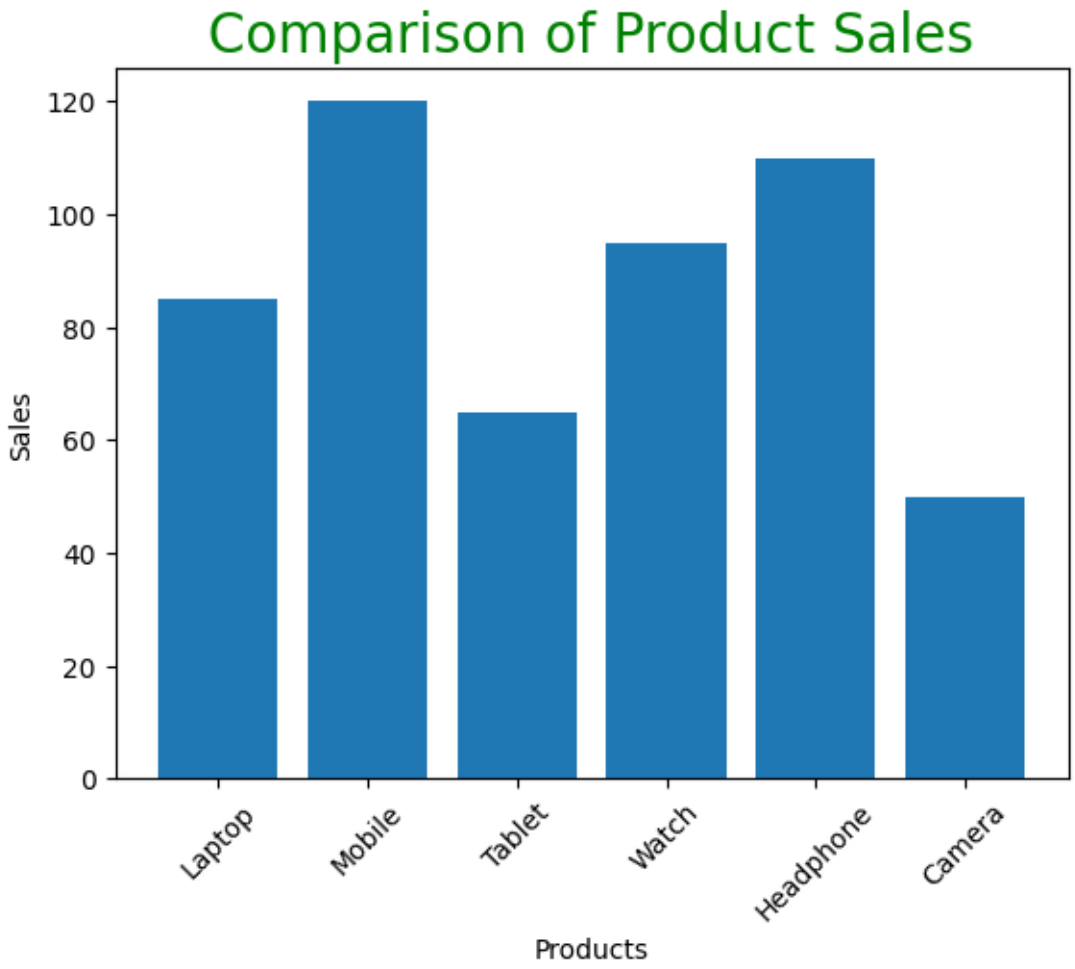
sales.xlabel("Products")

sales.ylabel("Sales")

sales.title("Comparison of Product Sales", fontsize=20, color="green")


sales.show()
```

OUTPUT:



RESULT:

The sales data of different products was successfully visualized using a bar chart, making it easy to compare the sales value of each product.

EXPERIMENT 3(c)

AIM:

To perform data visualization using a pie chart to represent the distribution of different expenses.

CODE:

```
import matplotlib.pyplot as expense

# Household expense data
labels = ['Food', 'Travel', 'Education', 'Shopping', 'Others']
Expenses = [3000, 2000, 4000, 2500, 1500]

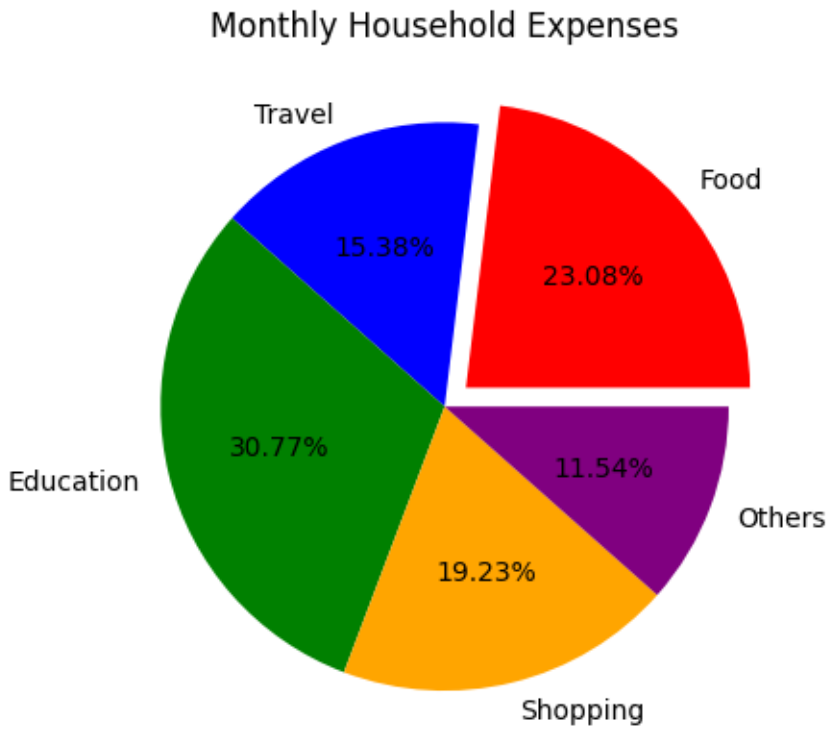
colors = ['red', 'blue', 'green', 'orange', 'purple']
explode = (0.1, 0, 0, 0, 0)

# Plotting the pie chart
expense.pie(Expenses, labels=labels, colors=colors,
        explode=explode, autopct='%0.2f%%')

expense.title('Monthly Household Expenses')

expense.show()
```

OUTPUT:



RESULT:

The distribution of monthly household expenses was successfully visualized using a pie chart, showing the percentage contribution of each expense category.

EXPERIMENT 4

AIM:

To count the frequency of occurrence of words in a given body of text using Python.

CODE:

```
import nltk

from nltk.tokenize import word_tokenize

from nltk.corpus import Gutenberg


nltk.download('gutenberg')

nltk.download('punkt')


sample = gutenberg.raw("carroll-alice.txt")


token = word_tokenize(sample)


wlist = []

for i in range(50):

   wlist.append(token[i])


wordfreq = [wlist.count(w) for w in wlist]


# Print the word-frequency pairs

print("Pairs\n" + str(list(zip(wlist, wordfreq))))
```

OUTPUT:

Pairs

[('[', 1), ('Alice', 2), ("'s", 1), ('Adventures', 1), ('in', 1), ('Wonderland', 1), ('by', 2), ('Lewis', 1), ('Carroll', 1), ('1865', 1), (']', 1), ('CHAPTER', 1), ('I', 1), ('.', 1), ('Down', 1), ('the', 3), ('Rabbit-Hole', 1), ('Alice', 2), ('was', 1), ('beginning', 1), ('to', 2), ('get', 1), ('very', 1), ('tired', 1), ('of', 2), ('sitting', 1), ('by', 2), ('her', 2), ('sister', 1), ('on', 1), ('the', 3), ('bank', 1), (',', 1), ('and', 1), ('of', 2), ('having', 1), ('nothing', 1), ('to', 2), ('do', 1), (':', 1), ('once', 1), ('or', 1), ('twice', 1), ('she', 1), ('had', 1), ('peeped', 1), ('into', 1), ('the', 3), ('book', 1), ('her', 2)]

RESULT:

The frequency of occurrence of words in the given text was successfully calculated using Python and displayed as word-frequency pairs.

EXPERIMENT 5

AIM:

To collect, load, and perform initial exploration of this Iris dataset using Python and data visualization libraries such as Pandas, Matplotlib and Seaborn.

CODE:

```
import pandas as pd

import numpy as np

import matplotlib.pyplot as plt

import seaborn as sns


from google.colab import files

uploaded = files.upload()


df = pd.read_csv("iris_dataset.csv")


print("First five rows of the dataset:")

print(df.head())


print("\nDataset Information:")

print(df.info())


print("\nSummary Statistics:")

print(df.describe())


df.hist(bins=20, figsize=(12, 8))
```

```
plt.tight_layout()
plt.show()

sns.pairplot(df, hue="Species")
plt.show()
```

OUTPUT:

First five rows of the dataset:

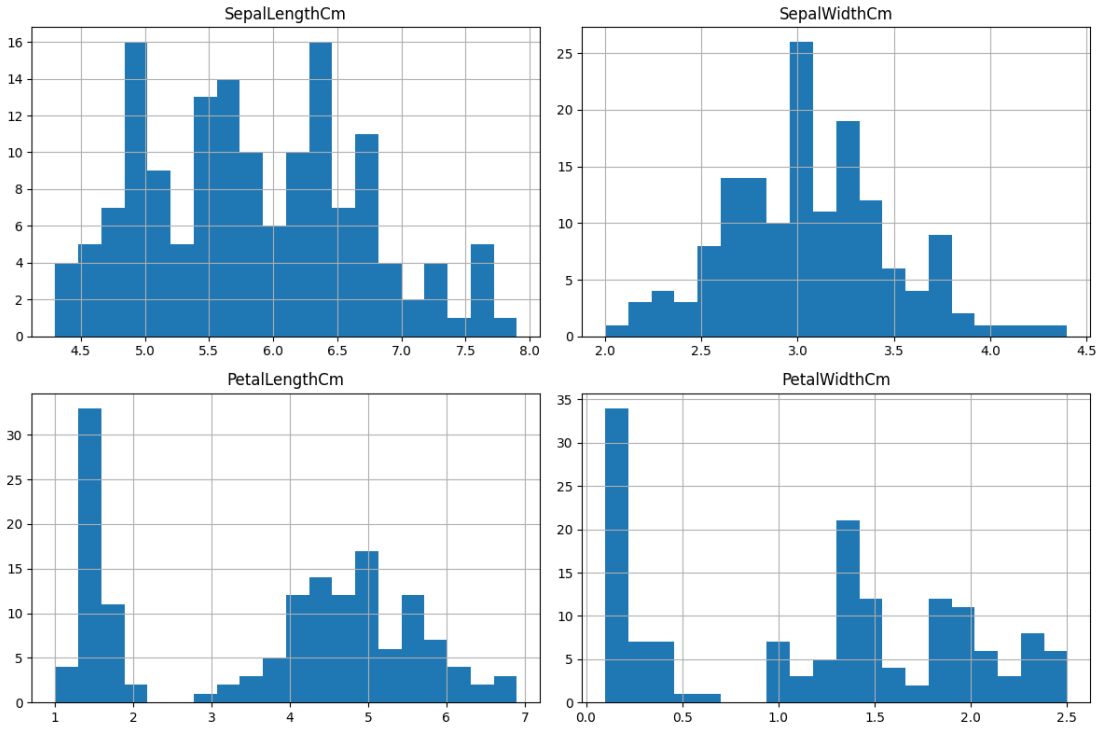
	SepalLengthCm	SepalWidthCm	PetalLengthCm	PetalWidthCm	Species
0	5.1	3.5	1.4	0.2	setosa
1	4.9	3.0	1.4	0.2	setosa
2	4.7	3.2	1.3	0.2	setosa
3	4.6	3.1	1.5	0.2	setosa
4	5.0	3.6	1.4	0.2	setosa

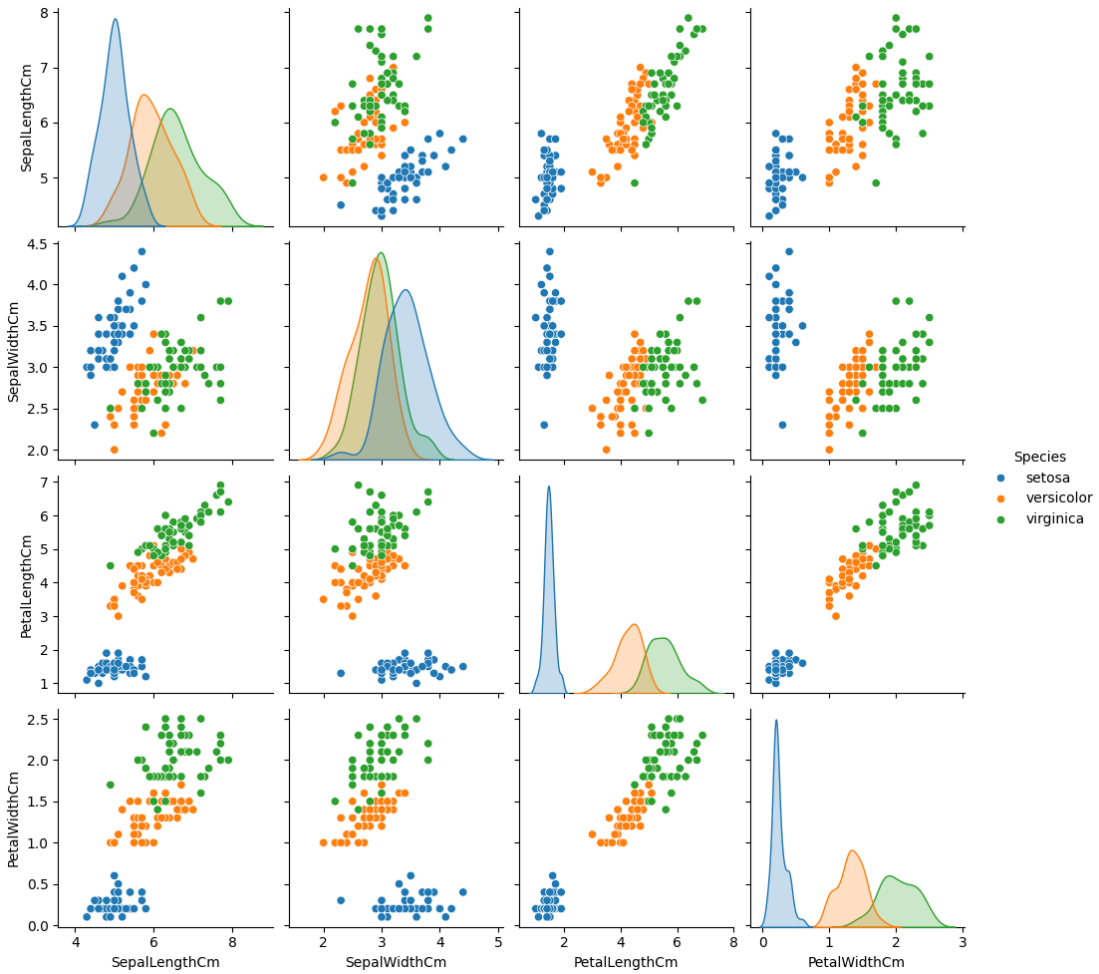
Dataset Information:

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 150 entries, 0 to 149
Data columns (total 5 columns):
 #   Column         Non-Null Count  Dtype
---  ------         --------------  -----
 0   SepalLengthCm  150 non-null    float64
```

```
 1   SepalWidthCm   150 non-null    float64
 2   PetalLengthCm  150 non-null    float64
 3   PetalWidthCm   150 non-null    float64
 4   Species        150 non-null    object
dtypes: float64(4), object(1)
memory usage: 6.0+ KB
None


Summary Statistics:
       SepalLengthCm  SepalWidthCm  PetalLengthCm  PetalWidthCm
count     150.000000    150.000000     150.000000    150.000000
mean        5.843333      3.057333       3.758000      1.199333
std         0.828066      0.435866       1.765298      0.762238
min         4.300000      2.000000       1.000000      0.100000
25%         5.100000      2.800000       1.600000      0.300000
50%         5.800000      3.000000       4.350000      1.300000
75%         6.400000      3.300000       5.100000      1.800000
max         7.900000      4.400000       6.900000      2.500000
```


SepalLengthCm
SepalWidthCm
PetalLengthCm
PetalWidthCm


SepalLengthCm
SepalWidthCm
PetalLengthCm
PetalWidthCm
Species
setosa
versicolor
virginica

RESULT:

The Iris dataset was successfully loaded and explored. The dataset structure, summary statistics, feature distributions, and relationships between features were visualized using Pandas, Matplotlib, and Seaborn.

EXPERIMENT 6

AIM:

To analyze daily website traffic data using time series techniques to identify trends, seasonality, and anomalies.

CODE:

```
import pandas as pd

import matplotlib.pyplot as plt

from statsmodels.tsa.seasonal import seasonal_decompose


data = pd.read_csv("website_traffic_timeseries.csv", parse_dates=["Date"], index_col="Date")


data.interpolate(method="linear", inplace=True)


print(data.head())


print(data.info())

print(data.describe())



from statsmodels.tsa.seasonal import seasonal_decompose


decomposition = seasonal_decompose(

    data["Page_Views"],

    model="additive",
```

```
    period=7
)

decomposition.plot()

plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
# Moving Average
data["Page_Views_MA"] = data["Page_Views"].rolling(window=30).mean()

data[["Page_Views", "Page_Views_MA"]].plot(figsize=(12, 5))
plt.title("Page Views with Moving Average")
plt.xlabel("Date")
plt.ylabel("Page Views")
plt.show()

# Anomaly Detection using Z-score
data["Page_Views_Z"] = (
    (data["Page_Views"] - data["Page_Views"].mean())
    / data["Page_Views"].std()
)

anomalies = data[data["Page_Views_Z"].abs() > 3]
```

```
print("Anomalies:")
print(anomalies[["Page_Views", "Page_Views_Z"]])


plt.figure(figsize=(12, 5))
plt.plot(data.index, data["Page_Views"], label="Page Views")
plt.scatter(
    anomalies.index,
    anomalies["Page_Views"],
    color="red",
    label="Anomalies"
)
plt.title("Website Traffic Anomaly Detection")
plt.xlabel("Date")
plt.ylabel("Page Views")
plt.legend()
plt.show()
```

OUTPUT:

	Page_Views	Unique_Visitors	Bounce_Rate
Date			
2025-01-01	2074	1280	48.91
2025-01-02	2326	1306	44.98
2025-01-03	2999	1678	45.09
2025-01-04	3185	1863	45.93
2025-01-05	2586	1555	50.24

<class 'pandas.core.frame.DataFrame'>

DatetimeIndex: 365 entries, 2025-01-01 to 2025-12-31

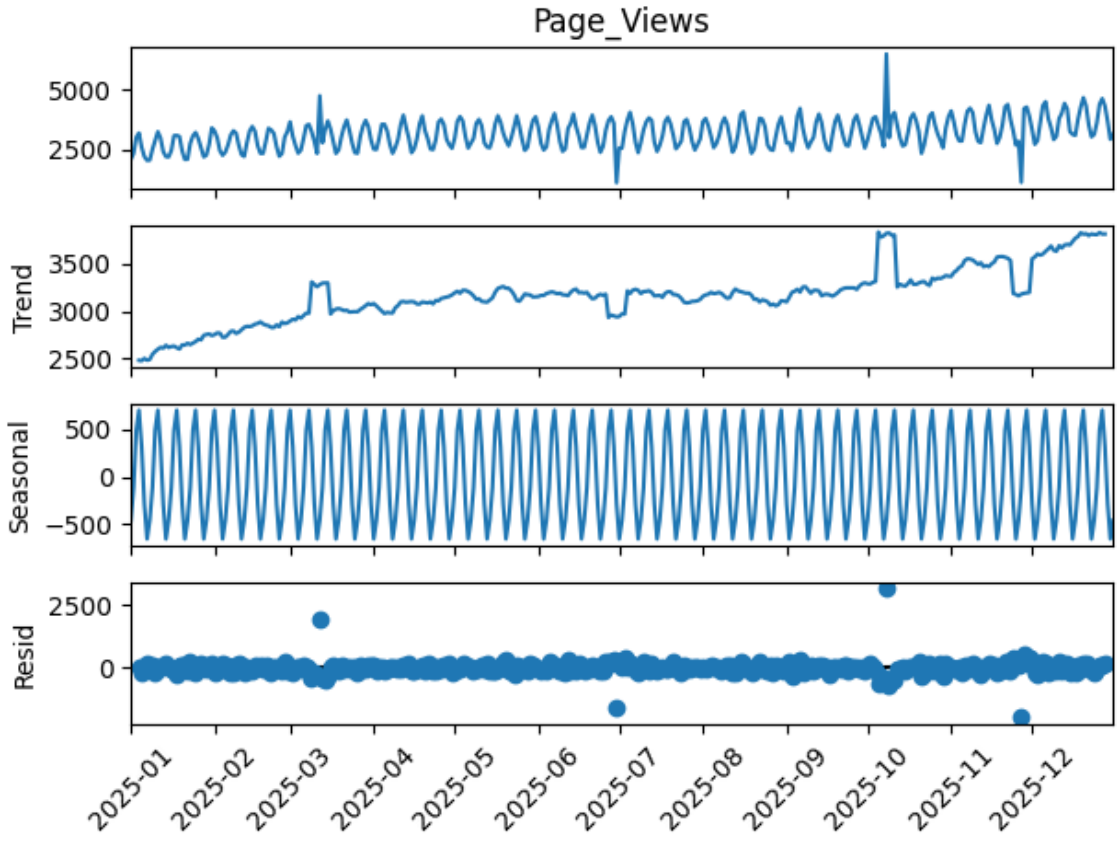
Data columns (total 3 columns):

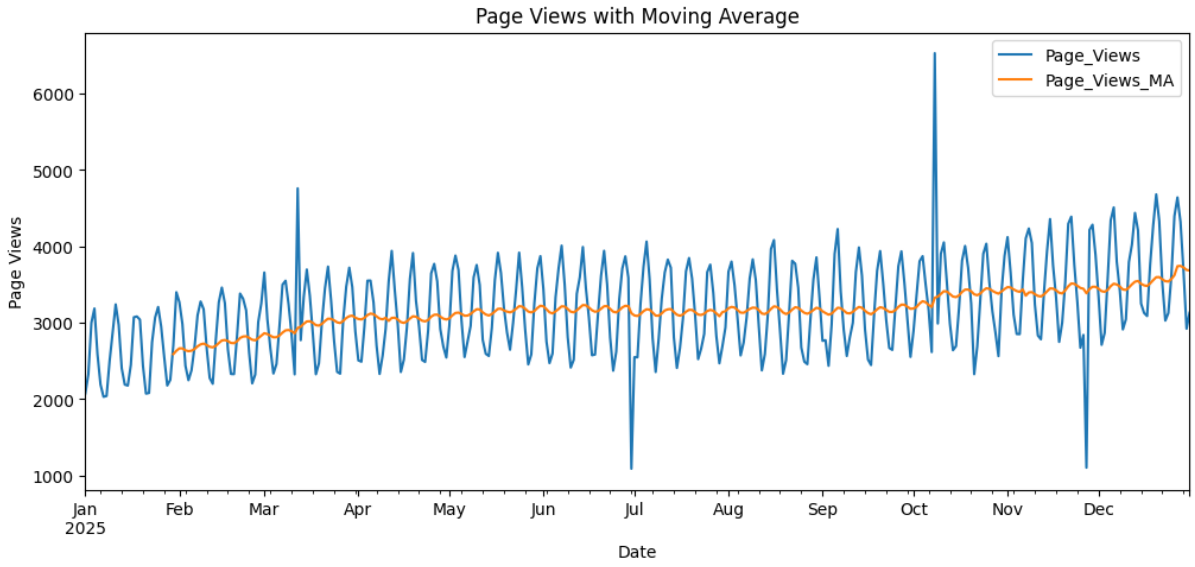
#	Column	Non-Null Count	Dtype
0	Page_Views	365 non-null	int64
1	Unique_Visitors	365 non-null	int64
2	Bounce_Rate	365 non-null	float64

dtypes: float64(1), int64(2)

memory usage: 11.4 KB

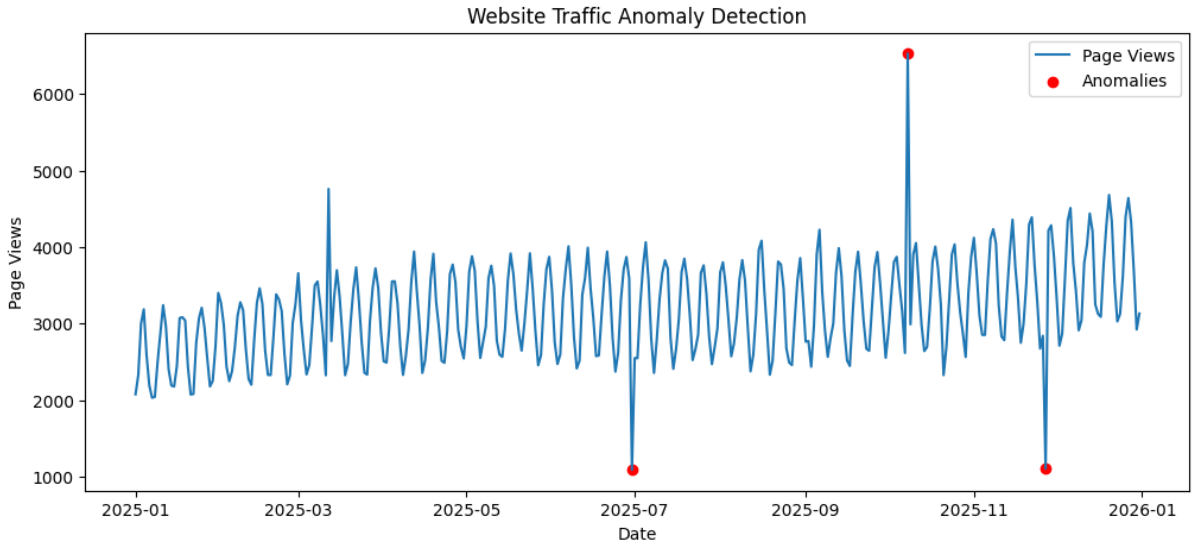
	Page_Views	Unique_Visitors	Bounce_Rate
count	365.000000	365.000000	365.000000
mean	3166.630137	1830.115068	48.036904
std	623.902005	371.898955	3.479127
min	1087.000000	617.000000	37.530000
25%	2686.000000	1550.000000	45.310000
50%	3105.000000	1797.000000	48.240000
75%	3614.000000	2083.000000	50.680000
max	6529.000000	3764.000000	60.550000





Anomalies:

Date	Page_Views	Page_Views_Z
2025-06-30	1087	-3.333264
2025-10-08	6529	5.389260
2025-11-27	1101	-3.310825



RESULT:

The website traffic data was successfully analyzed, and trends, seasonal patterns, moving averages, and anomalies were identified.

EXPERIMENT 7

AIM:

To explore random sampling and understand the concept of sampling distribution using Python.

CODE:

```
import numpy as np
import matplotlib.pyplot as plt


# Step 1: Generate a population
population_mean = 100
population_std = 15
population_size = 50000


population = np.random.normal(
    population_mean,
    population_std,
    population_size
)


# Step 2: Random sampling
sample_sizes = [25, 50, 100]
num_samples = 500


sample_means = {}
```

```
for size in sample_sizes:
    sample_means[size] = [ ]

    for _ in range(num_samples):
        sample = np.random.choice(
            population,
            size=size,
            replace=False
        )
        sample_means[size].append(np.mean(sample))

# Step 3: Plot sampling distributions
plt.figure(figsize=(12, 8))

for i, size in enumerate(sample_sizes):
    plt.subplot(3, 1, i + 1)

    plt.hist(
        sample_means[size],
        bins=30,
        alpha=0.7,
        label=f'Sample Size {size}'
    )

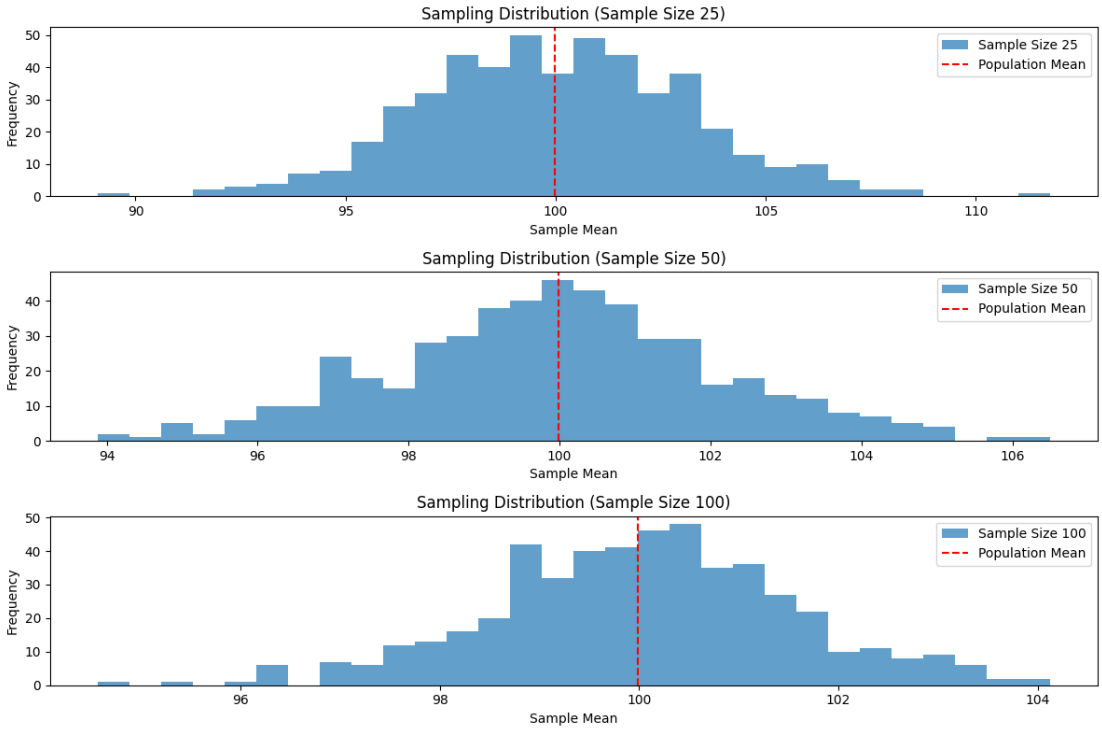
    plt.axvline(
```

```
        np.mean(population),
        color='red',
        linestyle='--',
        linewidth=1.5,
        label='Population Mean'
    )

    plt.title(f'Sampling Distribution (Sample Size {size})')
    plt.xlabel('Sample Mean')
    plt.ylabel('Frequency')
    plt.legend()

plt.tight_layout()
plt.show()
```

OUTPUT:



RESULT:

Random samples and their sampling distributions were successfully generated. The results showed that larger sample sizes produce a narrower sampling distribution around the population mean.

EXPERIMENT 8

AIM:

To perform a **two-tailed Z-test** on the salary dataset to determine whether the average salary differs significantly from the specified population mean.

CODE:

```
import pandas as pd

import numpy as np

import scipy.stats as stats


data = pd.read_excel("Salary_data.xlsx")


sample_data = data["Salary"].values


population_mean = 70000


sample_mean = np.mean(sample_data)

sample_std = np.std(sample_data, ddof=1)


n = len(sample_data)


z_statistic = (sample_mean - population_mean) / (sample_std / np.sqrt(n))


p_value = 2 * (1 - stats.norm.cdf(abs(z_statistic)))


print(f"Sample Mean: {sample_mean:.2f}")
```

```
print(f"Z-Statistic: {z_statistic:.4f}")
print(f"P-Value: {p_value:.4f}")

alpha = 0.05

if p_value < alpha:
    print("Reject the null hypothesis: The average salary is significantly different from ₹70,000.")
else:
    print("Fail to reject the null hypothesis: There is no significant difference in average salary from ₹70,000.")
```

OUTPUT:

```
Sample Mean: 76003.00
Z-Statistic: 1.1994
P-Value: 0.2304
Fail to reject the null hypothesis: There is no significant difference in average salary from ₹70,000.
```

RESULT:

The Z-test was successfully performed, and the result showed no significant difference between the average salary and the specified population mean.

EXPERIMENT 9

AIM:

To understand the use of the Matplotlib library in Data Science through visualization.

CODE:

```
import pandas as pd

import numpy as np

import seaborn as sns

import matplotlib.pyplot as plt


# Load dataset

data = pd.read_excel('/content/Social_Network_Ads.xlsx')

data


# Dataset information

data.info()


# Summary statistics

data.describe()


# Count values of Gender

data.value_counts('Gender')


# Count plot

sns.countplot(x='Gender', data=data)
```

```
plt.show()

# Convert Gender into dummy variables
dummies = pd.get_dummies(data.Gender)

FinalDataset = pd.concat(
    [dummies, data.iloc[:, [0, 2, 3, 4]]],
    axis=1
)

FinalDataset.head()

# Scatter plot 1
sns.scatterplot(
    x='Age',
    y='EstimatedSalary',
    hue='Gender',
    data=data
)
plt.show()

# Scatter plot 2
sns.scatterplot(
    x='Age',
    y='EstimatedSalary',
```

```
    hue='Purchased',
    data=data
)
plt.show()

# Pairplot
g = sns.pairplot(
    data[['Age', 'EstimatedSalary', 'Purchased']],
    hue='Purchased',
    height=3
)

# Rotate x-axis labels on all subplots
for ax in g.axes.flat:
    if ax is not None:
        ax.tick_params(axis='x', labelrotation=45)

plt.show()

# Histogram 1 - Estimated Salary
sns.FacetGrid(
    data,
    hue='Gender',
    height=5
).map(
```

```
    sns.histplot,
    'EstimatedSalary'
).add_legend()

plt.show()

# Histogram 2 - Age
sns.FacetGrid(
    data,
    hue='Gender',
    height=5
).map(
    sns.histplot,
    'Age'
).add_legend()

plt.show()
```

OUTPUT:

	User ID	Gender	Age	EstimatedSalary	Purchased
0	15624510	Male	19	19000	0
1	15810944	Male	35	20000	0
2	15668575	Female	26	43000	0
3	15603246	Female	27	57000	0
4	15804002	Male	19	76000	0
...	...	...	...	...	...
395	15691863	Female	46	41000	1
396	15706071	Male	51	23000	1
397	15654296	Female	50	20000	1
398	15755018	Male	36	33000	0
399	15594041	Female	49	36000	1

<class 'pandas.core.frame.DataFrame'>

RangeIndex: 400 entries, 0 to 399

Data columns (total 5 columns):

#	Column	Non-Null Count	Dtype
0	User ID	400 non-null	int64
1	Gender	400 non-null	object
2	Age	400 non-null	int64
3	EstimatedSalary	400 non-null	int64
4	Purchased	400 non-null	int64

dtypes: int64(4), object(1)

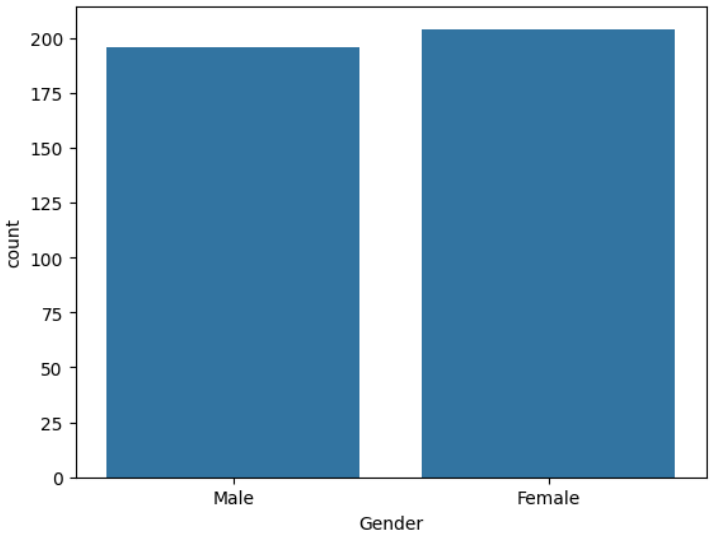
memory usage: 15.8+ KB

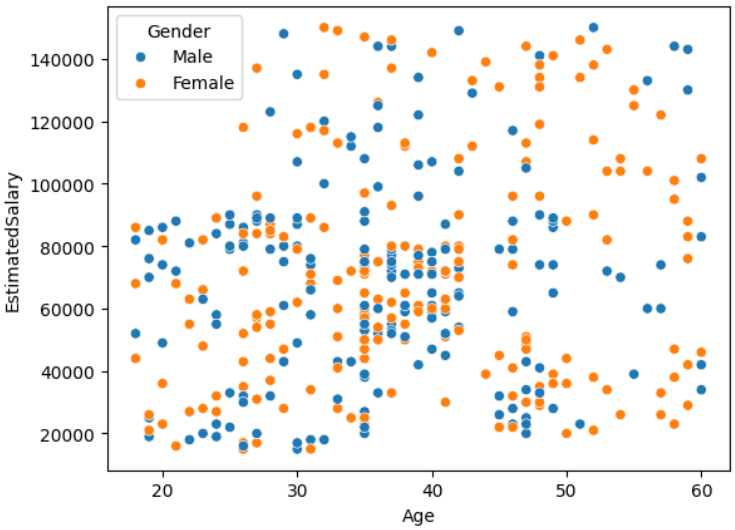
	User ID	Age	EstimatedSalary	Purchased
count	4.000000e+02	400.000000	400.000000	400.000000
mean	1.569154e+07	37.655000	69742.500000	0.357500
std	7.165832e+04	10.482877	34096.960282	0.479864
min	1.556669e+07	18.000000	15000.000000	0.000000
25%	1.562676e+07	29.750000	43000.000000	0.000000
50%	1.569434e+07	37.000000	70000.000000	0.000000
75%	1.575036e+07	46.000000	88000.000000	1.000000
max	1.581524e+07	60.000000	150000.000000	1.000000

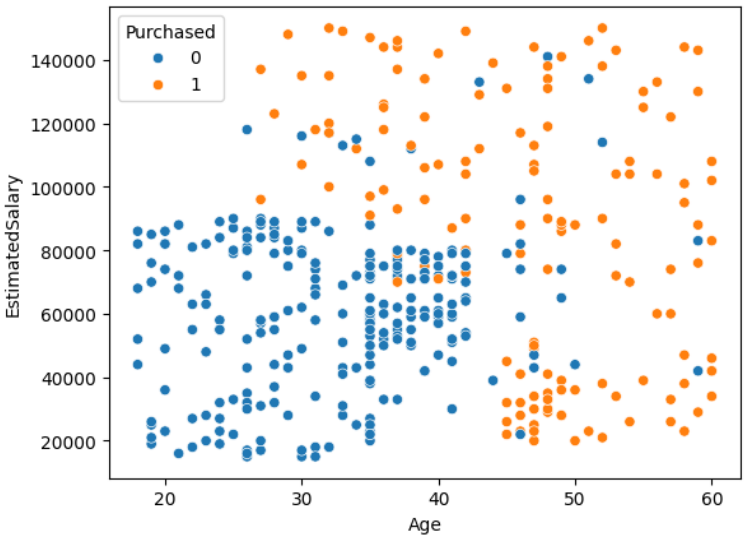
	count
Gender	
Female	204
Male	196

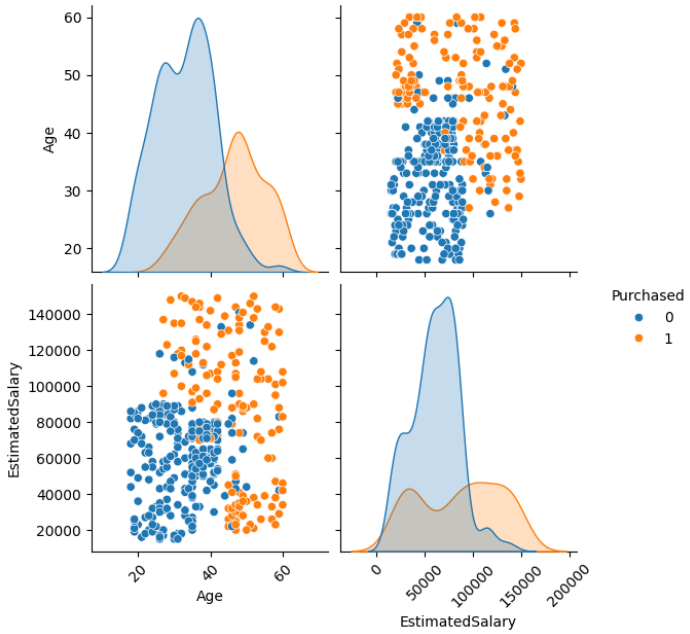
dtype: int64

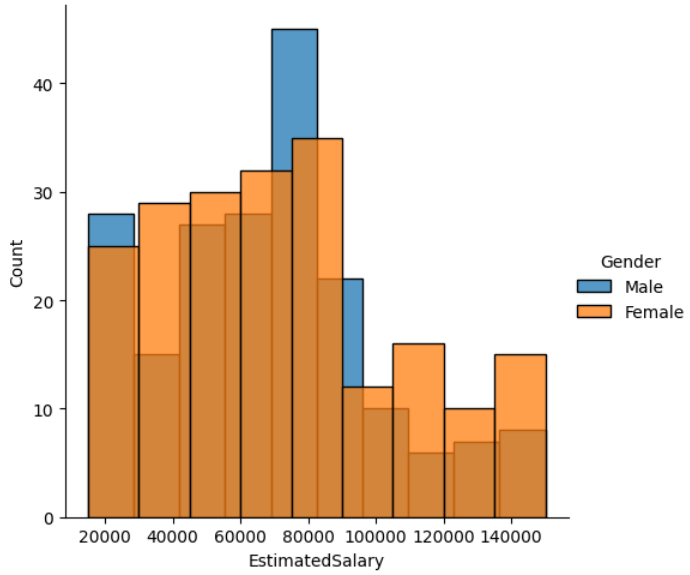
	Female	Male	User ID	Age	EstimatedSalary	Purchased
0	False	True	15624510	19	19000	0
1	False	True	15810944	35	20000	0
2	True	False	15668575	26	43000	0
3	True	False	15603246	27	57000	0
4	False	True	15804002	19	76000	0

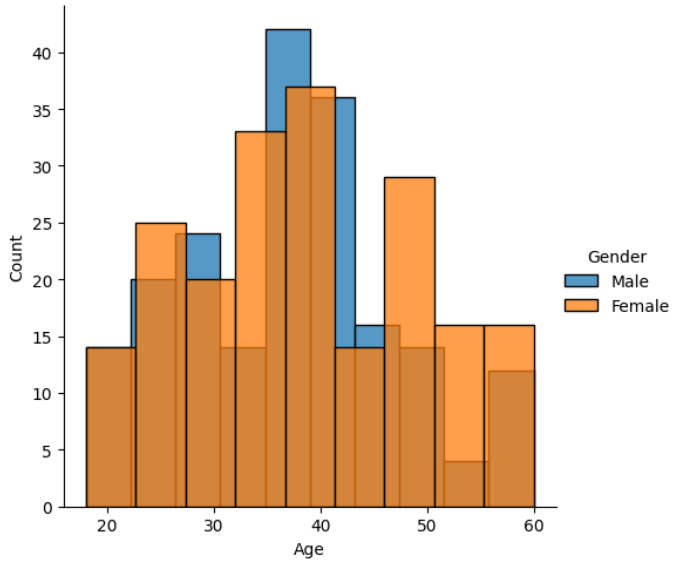

count
200
175
150
125
100
75
50
25
0
Male
Female
Gender


Gender
Male
Female
EstimatedSalary
140000
120000
100000
80000
60000
40000
20000
20
30
40
50
60
Age


Purchased
0
1
EstimatedSalary
140000
120000
100000
80000
60000
40000
20000
20
30
40
50
60
Age


Age
EstimatedSalary
Purchased
0
1
20
40
60
20000
40000
60000
80000
100000
120000
140000
0
50000
100000
150000
200000


Count
EstimatedSalary
Gender
Male
Female
0
10
20
30
40
20000
40000
60000
80000
100000
120000
140000


Count
Age
Gender
Male
Female
0
5
10
15
20
25
30
35
40
20
30
40
50
60

RESULT:

The Iris dataset was successfully visualized using different plots, helping to understand the distribution and relationships among the data features.

EXPERIMENT 10

AIM:

To understand and perform basic array functions using the NumPy library in Python.

CODE:

```
import numpy as np


# Create 1-D array using values from the dataset

array = np.array([15, 15, 16, 16, 17, 17, 18, 18, 19])


print("1. Original Array:")

print(array)


# Square root

print("\n2. Square Root of Array:")

print(np.sqrt(array))


# Number of dimensions

print("\n3. Number of Dimensions:")

print(array.ndim)


# Reshape into 3x3

new_array = array.reshape(3, 3)


print("\n4. Reshaped Array:")
```

```
print(new_array)

# Number of dimensions after reshape
print("\n5. Number of Dimensions after Reshape:")
print(new_array.ndim)

# Ravel
print("\n6. Ravelled Array:")
print(new_array.ravel())

# Reshape again
newm = new_array.reshape(3, 3)

print("\n7. New Array:")
print(newm)

# Slicing operations
print("\n8. newm[2,1:3]:")
print(newm[2,1:3])

print("\n9. newm[1:2,1:3]:")
print(newm[1:2,1:3])

print("\n10. new_array[0:3,0:0]:")
print(new_array[0:3,0:0])
```

```
print("\n11. new_array[0:2,0:1]:")
print(new_array[0:2,0:1])

print("\n12. new_array[0:3,0:1]:")
print(new_array[0:3,0:1])

print("\n13. new_array[1:3]:")
print(new_array[1:3])
```

OUTPUT:

```
1. Original Array:
[15 15 16 16 17 17 18 18 19]

2. Square Root of Array:
[3.87298335 3.87298335 4.         4.         4.12310563 4.12310563
 4.24264069 4.24264069 4.35889894]

3. Number of Dimensions:
1

4. Reshaped Array:
[[15 15 16]
```

[16 17 17]

[18 18 19]]

5. Number of Dimensions after Reshape:

2

6. Ravelled Array:

[15 15 16 16 17 17 18 18 19]

7. New Array:

[[15 15 16]

[16 17 17]

[18 18 19]]

8. newm[2,1:3]:

[18 19]

9. newm[1:2,1:3]:

[[17 17]]

10. new_array[0:3,0:0]:

[]

11. new_array[0:2,0:1]:

[[15]

[16]]

12. new_array[0:3,0:1]:

[[15]

[16]

[18]]

13. new_array[1:3]:

[[16 17 17]

[18 18 19]]

RESULT:

The NumPy array functions were successfully performed, and the array dimensions, reshaped arrays, and slicing operations were obtained.

EXPERIMENT 11

AIM:

To understand the creation, loading, manipulation, and basic statistical operations on DataFrames using the Pandas library.

CODE:

```
import numpy as np

import pandas as pd


# Create a DataFrame

list1 = [

   [1, "Arun", 85],

   [2, "Priya", 92]

]


df = pd.DataFrame(list1)

df.columns = ["ID", "Name", "Marks"]


print("Initial DataFrame:")

print(df)


# Display DataFrame information

df.info()


# Load student dataset from CSV file

df = pd.read_csv("student.csv")
```

```
print("\nStudent Dataset:")
print(df)

# Display DataFrame information
df.info()

# Display first five rows
print("\nFirst Five Rows:")
print(df.head())

# Display last five rows
print("\nLast Five Rows:")
print(df.tail())

# Display Marks column
print("\nMarks Column:")
print(df["Marks"])

# Display the type of Marks column
print("\nType of Marks Column:")
print(type(df["Marks"]))

# Calculate statistical measures
print("\nMean:")
```

```
print(df["Marks"].mean())

print("\nMedian:")
print(df["Marks"].median())

print("\nMode:")
print(df["Marks"].mode())

print("\nVariance:")
print(df["Marks"].var())

print("\nStandard Deviation:")
print(df["Marks"].std())

# Display descriptive statistics
print("\nDescriptive Statistics:")
print(df.describe())

print("\nDescriptive Statistics Including All Columns:")
print(df.describe(include="all"))

# Display column names
empcol = df.columns
print("\nColumn Names:")
print(empcol)
```

```
# Convert DataFrame into a NumPy array
emparray = df.values
print("\nDataFrame Values:")
print(emparray)

# Recreate DataFrame using array and column names
student_df = pd.DataFrame(
    emparray,
    columns=empcol
)

print("\nRecreated DataFrame:")
print(student_df)
```

OUTPUT:

1. Student Dataset

Output of reading student.csv:

Index	Name	Age	Marks	Department
0	Arun	19	85	CSE
1	Priya	20	92	CSE
2	Ravi	19	76	ECE
3	Divya	20	88	EEE
4	Kiran	19	95	CSE
5	Meena	20	81	ECE
6	Ajay	19	73	EEE
7	Sneha	20	90	CSE

2. Dataset Information (df.info())

RangeIndex: 8 entries, 0 to 7

Data columns (total 4 columns):

Name — 8 non-null — object

Age — 8 non-null — int64

Marks — 8 non-null — int64

Department — 8 non-null — object

dtypes: int64(2), object(2)

memory usage: 388.0+ bytes

3. First Five and Last Five Records

df.head():

Index	Name	Age	Marks	Department
0	Arun	19	85	CSE
1	Priya	20	92	CSE
2	Ravi	19	76	ECE
3	Divya	20	88	EEE
4	Kiran	19	95	CSE

df.tail():

Index	Name	Age	Marks	Department
3	Divya	20	88	EEE
4	Kiran	19	95	CSE
5	Meena	20	81	ECE
6	Ajay	19	73	EEE
7	Sneha	20	90	CSE

4. Selecting the Marks Column

Output of df["Marks"]:

Index	Marks
0	85
1	92
2	76
3	88
4	95
5	81
6	73
7	90

5. Descriptive Statistics for Marks

Statistic	Output
Mean	85.0
Median	86.5
Mode	73, 76, 81, 85, 88, 90, 92, 95 (all occur once)
Variance	60.57142857142857
Standard deviation	7.782764841072135

Output of df.describe():

Statistic	Age	Marks
count	8.000000	8.000000
mean	19.500000	85.000000
std	0.534522	7.782765
min	19.000000	73.000000
25%	19.000000	79.750000
50%	19.500000	86.500000
75%	20.000000	90.500000
max	20.000000	95.000000

6. Summary of All Columns (df.describe(include='all'))

Statistic	Name	Age	Marks	Department
count	8	8.000000	8.000000	8
unique	8	NaN	NaN	3
top	Arun	NaN	NaN	CSE
freq	1	NaN	NaN	4
mean	NaN	19.500000	85.000000	NaN
std	NaN	0.534522	7.782765	NaN
min	NaN	19.000000	73.000000	NaN
25%	NaN	19.000000	79.750000	NaN
50%	NaN	19.500000	86.500000	NaN
75%	NaN	20.000000	90.500000	NaN
max	NaN	20.000000	95.000000	NaN

7. Column Names

Output of df.columns:

Index(['Name', 'Age', 'Marks', 'Department'], dtype='object')

8. Dataset as a NumPy Array

Output of df.values (dtype=object):

```
[['Arun', 19, 85, 'CSE'],
 ['Priya', 20, 92, 'CSE'],
 ['Ravi', 19, 76, 'ECE'],
 ['Divya', 20, 88, 'EEE'],
 ['Kiran', 19, 95, 'CSE'],
 ['Meena', 20, 81, 'ECE'],
 ['Ajay', 19, 73, 'EEE'],
 ['Sneha', 20, 90, 'CSE']]
```

9. Reconstructed DataFrame

Output of student_df:

Index	Name	Age	Marks	Department
0	Arun	19	85	CSE
1	Priya	20	92	CSE
2	Ravi	19	76	ECE
3	Divya	20	88	EEE
4	Kiran	19	95	CSE
5	Meena	20	81	ECE
6	Ajay	19	73	EEE
7	Sneha	20	90	CSE

RESULT:

The Pandas library was successfully used to create, manipulate, and explore a DataFrame and perform basic statistical operations.

EXPERIMENT 12

AIM:

To detect outliers in a given dataset using the Interquartile Range (IQR) method.

CODE:

```
import numpy as np

import pandas as pd

import seaborn as sns

import matplotlib.pyplot as plt


# Load the dataset

df = pd.read_csv("student_marks_outliers.csv")

print(df)


# Extract Marks as an array

array = df["Marks"].values

print(array)


# Calculate mean

print("Mean:", array.mean())


# Calculate percentiles

print("25th Percentile:", np.percentile(array, 25))

print("50th Percentile:", np.percentile(array, 50))

print("75th Percentile:", np.percentile(array, 75))
```

```
print("100th Percentile:", np.percentile(array, 100))

# Calculate Q1, Q3 and IQR
Q1 = np.percentile(array, 25)
Q3 = np.percentile(array, 75)
IQR = Q3 - Q1

print("Q1 =", Q1)
print("Q3 =", Q3)
print("IQR =", IQR)

# Calculate lower and upper limits
lower_limit = Q1 - (1.5 * IQR)
upper_limit = Q3 + (1.5 * IQR)

print("Lower Limit:", lower_limit)
print("Upper Limit:", upper_limit)

# Detect outliers
outliers = array[
    (array < lower_limit) | (array > upper_limit)
]

print("Outliers:")
print(outliers)
```

```
# Define outlier detection function
def outDetection(array):
    Q1 = np.percentile(array, 25)
    Q3 = np.percentile(array, 75)
    IQR = Q3 - Q1

    lr = Q1 - (1.5 * IQR)
    ur = Q3 + (1.5 * IQR)

    return lr, ur

# Get lower and upper ranges
lr, ur = outDetection(array)

print("Lower Range:", lr)
print("Upper Range:", ur)

# Plot distribution before removing outliers
sns.histplot(array, bins=8, kde=True)
plt.title("Distribution of Student Marks")
plt.xlabel("Marks")
plt.ylabel("Count")
plt.show()
```

```
# Remove outliers
new_array = array[
    (array >= lr) & (array <= ur)
]

print("Data after removing outliers:")
print(new_array)

# Plot distribution after removing outliers
sns.histplot(new_array, bins=8, kde=True)
plt.title("Distribution After Removing Outliers")
plt.xlabel("Marks")
plt.ylabel("Count")
plt.show()
```

OUTPUT:

1. Original Student Marks Dataset

Output of reading student_marks_outliers.csv:

Index	Marks
0	72
1	68
2	75
3	81
4	77
5	69
6	85
7	74
8	80
9	78
10	83
11	71
12	76
13	88
14	79
15	82
16	73
17	86
18	90
19	84
20	67
21	70
22	92
23	87
24	89
25	94
26	66
27	91
28	95
29	76
30	10
31	15
32	130

2. Marks Array and Mean

Output of array.mean():

```
np.float64(77.36363636363636)
```

The marks were stored as a NumPy array. The separate full array printout is omitted because it repeats the dataset above.

3. Percentiles

Percentile	Output
25th Percentile	72.0
50th Percentile	79.0
75th Percentile	87.0
100th Percentile	130.0

4. IQR Calculation

Measure	Output
Q1	72.0
Q3	87.0
IQR = Q3 − Q1	15.0

5. Lower and Upper Limits

Limit	Output
Lower Limit	49.5
Upper Limit	109.5

6. Detected Outliers

Output:

Outliers:
[10 15 130]

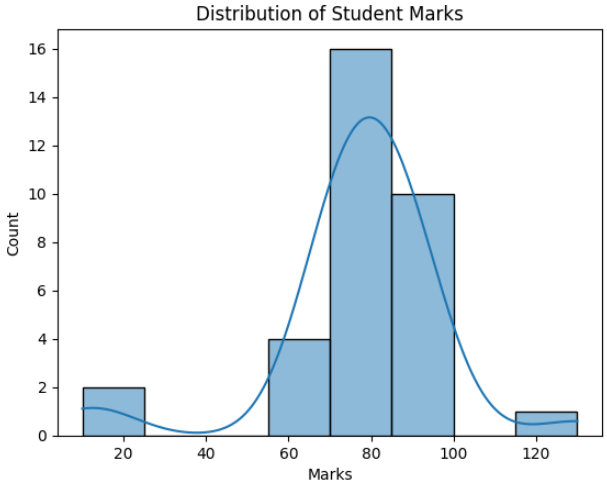
7. User-Defined Outlier Detection Function

The function outDetection(array) returns the lower and upper ranges. Output:

Lower Range: 49.5
Upper Range: 109.5

8. Distribution of Student Marks Before Removing Outliers

Histogram with KDE:



9. Marks After Removing Outliers

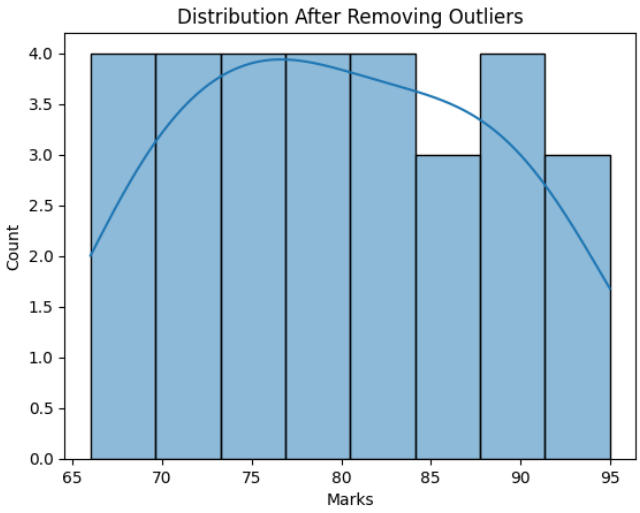
The values within the lower and upper limits were retained:

array([72, 68, 75, 81, 77, 69, 85, 74, 80, 78, 83, 71, 76, 88,
       79, 82, 73, 86, 90, 84, 67, 70, 92, 87, 89, 94, 66, 91, 95, 76])

The outlier values 10, 15, and 130 are excluded from the cleaned array.

10. Distribution After Removing Outliers

Histogram with KDE after outlier removal:



RESULT:

Outliers in the dataset were successfully identified using the IQR method, and the effect of removing them was visualised.

EXPERIMENT 13

AIM:

To handle missing, duplicate, and inappropriate data in a given dataset using the Pandas library.

CODE:

```
import numpy as np

import pandas as pd


# Load the dataset

df = pd.read_csv("student_missing_inappropriate.csv")

print(df)


# Identify duplicate records

print(df.duplicated())


# Display dataset information

df.info()


# Remove duplicate records

df.drop_duplicates(inplace=True)

print(df)


# Display number of records

print("Number of records:", len(df))
```

```
# Reset the index
index = np.array(list(range(0, len(df))))
df.set_index(index, inplace=True)
print(index)

# Display the dataset
print(df)

# Replace negative Marks with missing values
df.loc[df["Marks"] < 0, "Marks"] = np.nan
print(df)

# Replace negative Attendance with missing values
df.loc[df["Attendance"] < 0, "Attendance"] = np.nan
print(df)

# Replace negative Fees with missing values
df.loc[df["Fees"] < 0, "Fees"] = np.nan
print(df)

# Display unique departments
print(df["Department"].unique())

# Fill missing Marks using median
df["Marks"].fillna(
```

```
    round(df["Marks"].median()),
    inplace=True
)
print(df)

# Fill missing Attendance using median
df["Attendance"].fillna(
    round(df["Attendance"].median()),
    inplace=True
)
print(df)

# Fill missing Fees using mean
df["Fees"].fillna(
    round(df["Fees"].mean()),
    inplace=True
)
print(df)

# Display the final cleaned dataset
print(df)
```

OUTPUT:

1. Original Dataset

Index	StudentID	Name	Age	Marks	Dept.	Attend.	Fees
0	1	Arun	19	85	CSE	92	50000
1	2	Priya	20	92	CSE	88	52000
2	3	Ravi	19	76	ECE	75	48000
3	4	Divya	20	-5	EEE	91	51000
4	5	Kiran	21	88	CSE	86	53000
5	5	Kiran	21	88	CSE	86	53000
6	7	Meena	20	81	ECE	80	-1000
7	8	Ajay	19	95	EEE	95	50000
8	9	Sneha	20	-10	CSE	89	54000
9	10	Vijay	22	90	CSE	-1	55000

2. Duplicate Check

df.duplicated()
0 False
1 False
2 False
3 False
4 False
5 True
6 False
7 False
8 False
9 False
dtype: bool

The record at index 5 is a duplicate and is removed using df.drop_duplicates(inplace=True).

3. Dataset Information (df.info())

RangeIndex: 10 entries, 0 to 9
Data columns (total 7 columns):
StudentID 10 non-null int64
Name 10 non-null object
Age 10 non-null int64
Marks 10 non-null int64
Department 10 non-null object

Attendance 10 non-null int64
Fees   10 non-null int64
dtypes: int64(5), object(2)
memory usage: 692.0+ bytes

4. Dataset After Removing Duplicate

Index	StudentID	Name	Age	Marks	Dept.	Attend.	Fees
0	1	Arun	19	85	CSE	92	50000
1	2	Priya	20	92	CSE	88	52000
2	3	Ravi	19	76	ECE	75	48000
3	4	Divya	20	-5	EEE	91	51000
4	5	Kiran	21	88	CSE	86	53000
6	7	Meena	20	81	ECE	80	-1000
7	8	Ajay	19	95	EEE	95	50000
8	9	Sneha	20	-10	CSE	89	54000
9	10	Vijay	22	90	CSE	-1	55000

len(df) = 9

5. Replacing Inappropriate Negative Values with NaN

Negative values in Marks, Attendance, and Fees are treated as missing values. Output after applying the three replacements:

Index	StudentID	Name	Age	Marks	Dept.	Attend.	Fees
0	1	Arun	19	85.0	CSE	92.0	50000.0
1	2	Priya	20	92.0	CSE	88.0	52000.0
2	3	Ravi	19	76.0	ECE	75.0	48000.0
3	4	Divya	20	NaN	EEE	91.0	51000.0
4	5	Kiran	21	88.0	CSE	86.0	53000.0
5	7	Meena	20	81.0	ECE	80.0	NaN
6	8	Ajay	19	95.0	EEE	95.0	50000.0
7	9	Sneha	20	NaN	CSE	89.0	54000.0
8	10	Vijay	22	90.0	CSE	NaN	55000.0

6. Unique Departments

array(['CSE', 'ECE', 'EEE'], dtype=object)

7. Missing-Value Imputation

Column	Method Used	Replacement Value
Marks	Median, rounded	88
Attendance	Median, rounded	88
Fees	Mean, rounded	51625

The notebook fills missing Marks and Attendance with their rounded medians, and missing Fees with the rounded mean.

8. Final Cleaned Dataset

Index	StudentID	Name	Age	Marks	Dept.	Attend.	Fees
0	1	Arun	19	85.0	CSE	92.0	50000.0
1	2	Priya	20	92.0	CSE	88.0	52000.0
2	3	Ravi	19	76.0	ECE	75.0	48000.0
3	4	Divya	20	88.0	EEE	91.0	51000.0
4	5	Kiran	21	88.0	CSE	86.0	53000.0
5	7	Meena	20	81.0	ECE	80.0	51625.0
6	8	Ajay	19	95.0	EEE	95.0	50000.0
7	9	Sneha	20	88.0	CSE	89.0	54000.0
8	10	Vijay	22	90.0	CSE	88.0	55000.0

RESULT:

Duplicate records and inappropriate values were successfully identified and handled, and the dataset was cleaned using Pandas.

EXPERIMENT 14

AIM:

To understand and implement feature scaling techniques on a given dataset.

CODE:

```
import numpy as np
import pandas as pd

# Upload the dataset (Google Colab)
from google.colab import files
uploaded = files.upload()

# Load the dataset
df = pd.read_csv("feature_scaling_dataset.csv")
print(df)

# Handle missing values in Country
df["Country"] = df["Country"].fillna(df["Country"].mode()[0])
print(df)

# Separate features and label
features = df.iloc[:, :-1].values
label = df.iloc[:, -1].values
print(features)

# Import SimpleImputer
```

```
from sklearn.impute import SimpleImputer

# Create imputers
age_imputer = SimpleImputer(strategy="mean")
salary_imputer = SimpleImputer(strategy="mean")

# Fit the imputers
age_imputer.fit(features[:, [1]])
salary_imputer.fit(features[:, [2]])

# Replace missing values
features[:, [1]] = age_imputer.transform(features[:, [1]])
features[:, [2]] = salary_imputer.transform(features[:, [2]])
print(features)

# Apply One-Hot Encoding
from sklearn.preprocessing import OneHotEncoder

oh = OneHotEncoder(sparse_output=False)

Country = oh.fit_transform(features[:, [0]])
print(Country)

# Combine encoded and numerical features
final_set = np.concatenate(
```

```
    (Country, features[:, [1, 2]]),
    axis=1
)
print(final_set)

# Apply StandardScaler
from sklearn.preprocessing import StandardScaler

sc = StandardScaler()
sc.fit(final_set)

feat_standard_scaler = sc.transform(final_set)
print(feat_standard_scaler)

# Apply MinMaxScaler
from sklearn.preprocessing import MinMaxScaler

mms = MinMaxScaler(feature_range=(0, 1))
mms.fit(final_set)

feat_minmax_scaler = mms.transform(final_set)
print(feat_minmax_scaler)
```

OUTPUT:

1. Original Dataset

```
   Country   Age   Salary Purchased
0   France  42.0  68000.0        No
1    Spain  29.0  51000.0       Yes
2  Germany  31.0  57000.0        No
3    Spain  36.0  64000.0       Yes
4  Germany   NaN      NaN        No
5   France  34.0  61000.0       Yes
6    Spain   NaN  55000.0        No
7   France  46.0  82000.0       Yes
8  Germany  51.0  76000.0        No
9   France  39.0  70000.0       Yes
```

2. Features After Missing Values Are Filled

```
array([['France', 42.0, 68000.0],
       ['Spain', 29.0, 51000.0],
       ['Germany', 31.0, 57000.0],
       ['Spain', 36.0, 64000.0],
       ['Germany', 38.5, 64888.88888888889],
       ['France', 34.0, 61000.0],
       ['Spain', 38.5, 55000.0],
       ['France', 46.0, 82000.0],
       ['Germany', 51.0, 76000.0],
       ['France', 39.0, 70000.0]], dtype=object)
```

3. One-Hot Encoding

```
array([[1., 0., 0.],
       [0., 0., 1.],
       [0., 1., 0.],
       [0., 0., 1.],
       [0., 1., 0.],
       [1., 0., 0.],
       [0., 0., 1.],
       [1., 0., 0.],
       [0., 1., 0.],
       [1., 0., 0.]])
```

4. Final Dataset Before Scaling

```
array([[1.0, 0.0, 0.0, 42.0, 68000.0],
       [0.0, 0.0, 1.0, 29.0, 51000.0],
       [0.0, 1.0, 0.0, 31.0, 57000.0],
       [0.0, 0.0, 1.0, 36.0, 64000.0],
       [0.0, 1.0, 0.0, 38.5, 64888.88888888889],
       [1.0, 0.0, 0.0, 34.0, 61000.0],
       [0.0, 0.0, 1.0, 38.5, 55000.0],
       [1.0, 0.0, 0.0, 46.0, 82000.0],
       [0.0, 1.0, 0.0, 51.0, 76000.0],
       [1.0, 0.0, 0.0, 39.0, 70000.0]], dtype=object)
```

5. StandardScaler Output

```
array([[ 1.22474487, -0.65465367, -0.65465367,  0.5547873 ,  0.34337869],
       [-0.81649658, -0.65465367,  1.52752523, -1.50585123, -1.53294056],
       [-0.81649658,  1.52752523, -0.65465367, -1.18882992, -0.87071024],
       [-0.81649658, -0.65465367,  1.52752523, -0.39627664, -0.0981082 ],
       [-0.81649658,  1.52752523, -0.65465367,  0.        ,  0.        ],
       [ 1.22474487, -0.65465367, -0.65465367, -0.71329795, -0.42922336],
       [-0.81649658, -0.65465367,  1.52752523,  0.        , -1.09145368],
       [ 1.22474487, -0.65465367, -0.65465367,  1.18882992,  1.88858277],
       [-0.81649658,  1.52752523, -0.65465367,  1.9813832 ,  1.22635245],
       [ 1.22474487, -0.65465367, -0.65465367,  0.07925533,  0.56412213]])
```

6. MinMaxScaler Output

```
array([[1.        , 0.        , 0.        , 0.59090909, 0.5483871 ],
       [0.        , 0.        , 1.        , 0.        , 0.        ],
       [0.        , 1.        , 0.        , 0.09090909, 0.19354839],
       [0.        , 0.        , 1.        , 0.31818182, 0.41935484],
       [0.        , 1.        , 0.        , 0.43181818, 0.44802867],
       [1.        , 0.        , 0.        , 0.22727273, 0.32258065],
       [0.        , 0.        , 1.        , 0.43181818, 0.12903226],
       [1.        , 0.        , 0.        , 0.77272727, 1.        ],
       [0.        , 1.        , 0.        , 1.        , 0.80645161],
       [1.        , 0.        , 0.        , 0.45454545, 0.61290323]])
```

RESULT:

The dataset was successfully scaled using StandardScaler and MinMaxScaler.

EXPERIMENT 15

AIM:

To understand the importance of data preprocessing and handle missing values in a dataset.

CODE:

```
import numpy as np

import pandas as pd


# Load the dataset

df = pd.read_csv("/content/customer_purchase_dataset.csv")

print(df)


# Display dataset information

df.info()


# Find the mode of City

print(df.City.mode())

print(df.City.mode()[0])

print(type(df.City.mode()))


# Handle missing values in City

df.City.fillna(df.City.mode()[0], inplace=True)


# Handle missing values in Age using median

df.Age.fillna(df.Age.median(), inplace=True)


# Handle missing values in Income using mean
```

```
df.Income.fillna(round(df.Income.mean()), inplace=True)

# Display the dataset after handling missing values
print(df)

# Encode City using one-hot encoding
print(pd.get_dummies(df.City))

# Combine encoded City with selected columns
updated_dataset = pd.concat(
    [pd.get_dummies(df.City), df.iloc[:, [1, 2, 4, 5]]],
    axis=1
)

print(updated_dataset)

# Display updated dataset information
updated_dataset.info()

# Convert Purchased values into numerical values
updated_dataset.Purchased.replace(
    ["No", "Yes"], [0, 1], inplace=True
)

# Display the final preprocessed dataset
print(updated_dataset)
```

OUTPUT:

1. Original Customer Purchase Dataset

Index	Customer_ID	Gender	Age	City	Income	Purchased
0	301	Male	22.0	Chennai	25000.0	No
1	302	Female	25.0	Bangalore	40000.0	Yes
2	303	Male	28.0	Chennai	52000.0	Yes
3	304	Female	24.0	Hyderabad	NaN	No
4	305	Male	30.0	Bangalore	65000.0	Yes
5	306	Female	27.0	Chennai	48000.0	Yes
6	307	Male	NaN	Hyderabad	35000.0	No
7	308	Female	29.0	Bangalore	NaN	Yes
8	309	Male	32.0	Chennai	72000.0	Yes
9	310	Female	26.0	Hyderabad	42000.0	No

2. Dataset Information (df.info())

RangeIndex: 10 entries, 0 to 9
Data columns (total 6 columns):
Customer_ID 10 non-null int64
Gender 10 non-null object
Age 9 non-null float64
City 10 non-null object
Income 8 non-null float64
Purchased 10 non-null object
dtypes: float64(2), int64(1), object(3)
memory usage: 612.0+ bytes

3. Finding and Filling Missing Values

Column	Method	Value Used
City	Mode	Chennai
Age	Median	27.0
Income	Rounded mean	47375

Mode output: df.City.mode() returns Chennai. The notebook fills missing City values with the mode, missing Age with the median, and missing Income with the rounded mean.

4. Dataset After Filling Missing Values

Index	Customer_ID	Gender	Age	City	Income	Purchased
0	301	Male	22.0	Chennai	25000.0	No
1	302	Female	25.0	Bangalore	40000.0	Yes
2	303	Male	28.0	Chennai	52000.0	Yes
3	304	Female	24.0	Hyderabad	47375.0	No
4	305	Male	30.0	Bangalore	65000.0	Yes
5	306	Female	27.0	Chennai	48000.0	Yes
6	307	Male	27.0	Hyderabad	35000.0	No
7	308	Female	29.0	Bangalore	47375.0	Yes
8	309	Male	32.0	Chennai	72000.0	Yes
9	310	Female	26.0	Hyderabad	42000.0	No

5. One-Hot Encoding of City

Output of pd.get_dummies(df.City):

Index	Bangalore	Chennai	Hyderabad
0	False	True	False
1	True	False	False
2	False	True	False
3	False	False	True
4	True	False	False
5	False	True	False
6	False	False	True
7	True	False	False
8	False	True	False
9	False	False	True

6. Updated Dataset After One-Hot Encoding

Idx	Banga-lore	Chennai	Hydera-bad	Gender	Age	Income	Purchased
0	False	True	False	Male	22.0	25000.0	No
1	True	False	False	Female	25.0	40000.0	Yes
2	False	True	False	Male	28.0	52000.0	Yes
3	False	False	True	Female	24.0	47375.0	No
4	True	False	False	Male	30.0	65000.0	Yes
5	False	True	False	Female	27.0	48000.0	Yes
6	False	False	True	Male	27.0	35000.0	No
7	True	False	False	Female	29.0	47375.0	Yes
8	False	True	False	Male	32.0	72000.0	Yes
9	False	False	True	Female	26.0	42000.0	No

7. Updated Dataset Information

RangeIndex: 10 entries, 0 to 9
Data columns (total 7 columns):
Bangalore 10 non-null bool
Chennai 10 non-null bool
Hyderabad 10 non-null bool
Gender 10 non-null object
Age 10 non-null float64
Income 10 non-null float64
Purchased 10 non-null object
dtypes: bool(3), float64(2), object(2)
memory usage: 482.0+ bytes

8. Encoding Purchased as 0 and 1

The notebook replaces 'No' with 0 and 'Yes' with 1.

Idx	Banga-lore	Chennai	Hyder bad	Gender	Age	Income	Purchased
0	False	True	False	Male	22.0	25000.0	0
1	True	False	False	Female	25.0	40000.0	1
2	False	True	False	Male	28.0	52000.0	1
3	False	False	True	Female	24.0	47375.0	0
4	True	False	False	Male	30.0	65000.0	1
5	False	True	False	Female	27.0	48000.0	1
6	False	False	True	Male	27.0	35000.0	0
7	True	False	False	Female	29.0	47375.0	1
8	False	True	False	Male	32.0	72000.0	1
9	False	False	True	Female	26.0	42000.0	0

RESULT:

The given dataset was successfully preprocessed by handling missing values and preparing it for further analysis.

EXPERIMENT 16

AIM:

To perform basic data exploration and visualisation of the given dataset using Python.

CODE:

```
import seaborn as sns
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

# Load the dataset
df = pd.read_csv("/content/student_eda_dataset.csv")
print(df)

# Display first five rows
print(df.head())

# Distribution plot of Marks with KDE
sns.displot(df.Marks, kde=True)
plt.show()

# Distribution plot of Marks without KDE
sns.displot(df.Marks, kde=False)
plt.show()
```

```
# Joint plot of Study Hours and Marks
sns.jointplot(x=df.Study_Hours, y=df.Marks)
plt.show()

# Regression plot
sns.jointplot(x=df.Study_Hours, y=df.Marks, kind="reg")
plt.show()

# Hexagonal joint plot
sns.jointplot(x=df.Study_Hours, y=df.Marks, kind="hex")
plt.show()

# Pair plot of numerical features
sns.pairplot(
    df[['Study_Hours', 'Attendance', 'Marks', 'Assignment_Score']]
)
plt.show()

# Pair plot based on Study Mode
sns.pairplot(
    df[['Study_Hours', 'Attendance', 'Marks',
        'Assignment_Score', 'Study_Mode']],
    hue='Study_Mode'
)
```

```
plt.show()

# Pair plot based on Gender
sns.pairplot(
    df[['Study_Hours', 'Attendance', 'Marks',
        'Assignment_Score', 'Gender']],
    hue='Gender'
)
plt.show()

# Correlation heatmap
sns.heatmap(
    df[['Study_Hours', 'Attendance', 'Marks',
        'Assignment_Score']].corr(),
    annot=True
)
plt.show()

# Box plot of Marks
sns.boxplot(y=df.Marks)
plt.show()

# Box plot of Study Hours
sns.boxplot(y=df.Study_Hours)
plt.show()
```

```
# Count plot of Department
sns.countplot(x=df.Department)
plt.show()

# Count plot of Gender
sns.countplot(x=df.Gender)
plt.show()

# Pie chart of Gender
df.Gender.value_counts().plot(
    kind='pie', autopct='%1.1f%%'
)
plt.ylabel('')
plt.show()

# Bar chart of Gender
df.Gender.value_counts().plot(kind='bar')
plt.xlabel('Gender')
plt.ylabel('Count')
plt.show()

# Count plot of Pass status
sns.countplot(x=df.Passed)
plt.show()
```

```
# Count plot of Study Mode
sns.countplot(x=df.Study_Mode)
plt.show()

# Gender versus Pass status
sns.countplot(x=df.Gender, hue=df.Passed)
plt.show()

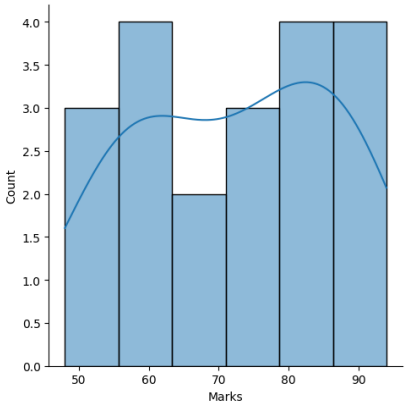
# Department versus Pass status
sns.countplot(x=df.Department, hue=df.Passed)
plt.show()

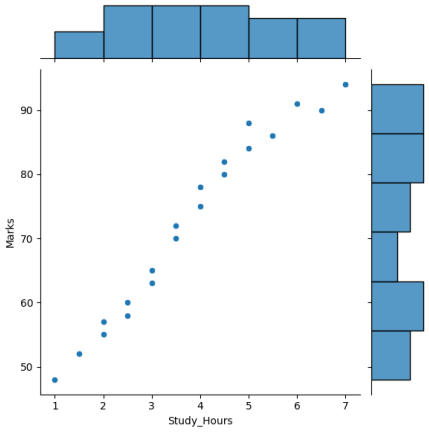
# Department distribution for Online study mode
sns.countplot(
    x=df[df.Study_Mode == 'Online']['Department']
)
plt.show()
```

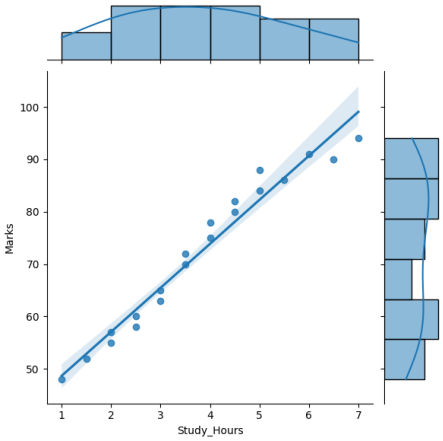
OUTPUT:

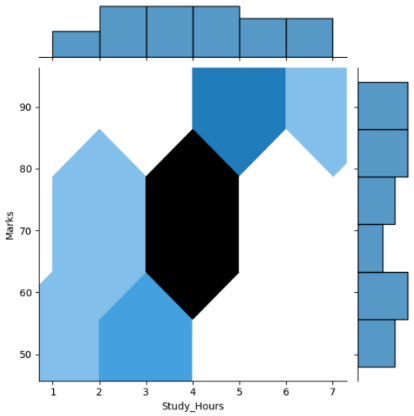
	Student_ID	Gender	Department	Study_Mode	Study_Hours	Attendance	Marks	Assignment_Score	Passed
0	101	Male	CSE	Online	2.5	72	58	60	No
1	102	Female	ECE	Offline	4.0	85	78	80	Yes
2	103	Male	CSE	Online	3.0	78	65	68	Yes
3	104	Female	IT	Offline	5.0	92	88	90	Yes
4	105	Male	ECE	Online	1.5	68	52	55	No
5	106	Female	CSE	Offline	6.0	95	91	93	Yes
6	107	Male	IT	Online	3.5	80	70	72	Yes
7	108	Female	ECE	Offline	4.5	88	82	85	Yes
8	109	Male	CSE	Online	2.0	70	55	58	No
9	110	Female	IT	Offline	5.5	90	86	88	Yes
10	111	Male	ECE	Offline	7.0	96	94	96	Yes
11	112	Female	CSE	Online	3.0	76	63	65	Yes
12	113	Male	IT	Offline	4.0	84	75	78	Yes
13	114	Female	ECE	Online	2.5	73	60	62	Yes
14	115	Male	CSE	Offline	6.5	94	90	92	Yes
15	116	Female	IT	Online	1.0	65	48	50	No
16	117	Male	ECE	Offline	5.0	89	84	87	Yes
17	118	Female	CSE	Online	3.5	82	72	75	Yes
18	119	Male	IT	Offline	2.0	71	57	59	No
19	120	Female	ECE	Online	4.5	87	80	83	Yes

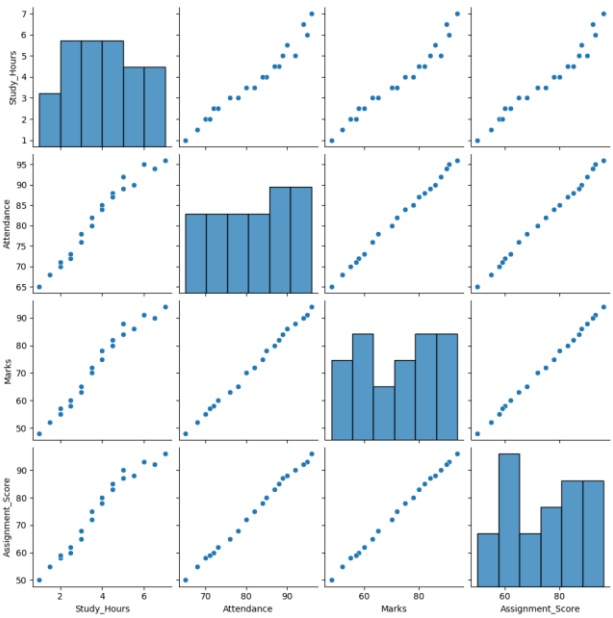
	Student_ID	Gender	Department	Study_Mode	Study_Hours	Attendance	Marks	Assignment_Score	Passed
0	101	Male	CSE	Online	2.5	72	58	60	No
1	102	Female	ECE	Offline	4.0	85	78	80	Yes
2	103	Male	CSE	Online	3.0	78	65	68	Yes
3	104	Female	IT	Offline	5.0	92	88	90	Yes
4	105	Male	ECE	Online	1.5	68	52	55	No

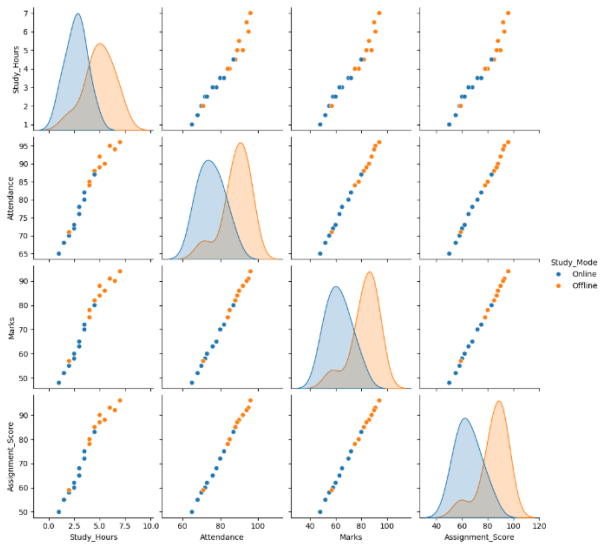

Count
Marks

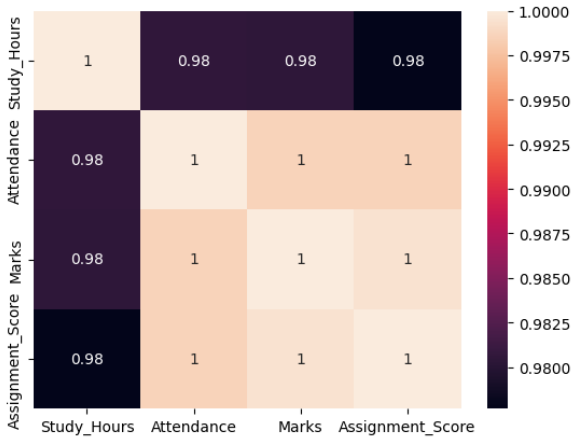

Marks
Study_Hours

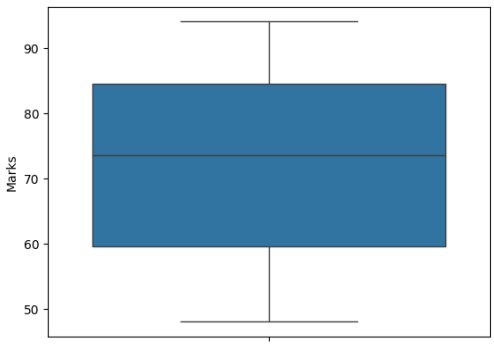

Marks
Study_Hours

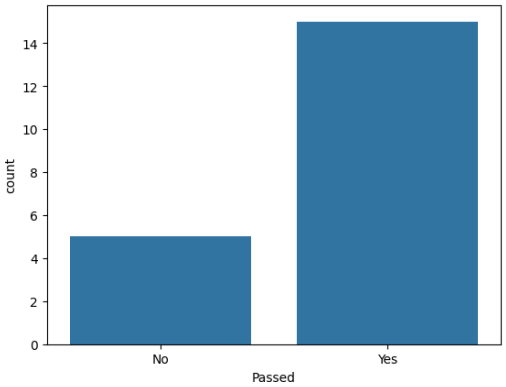

Marks
Study_Hours

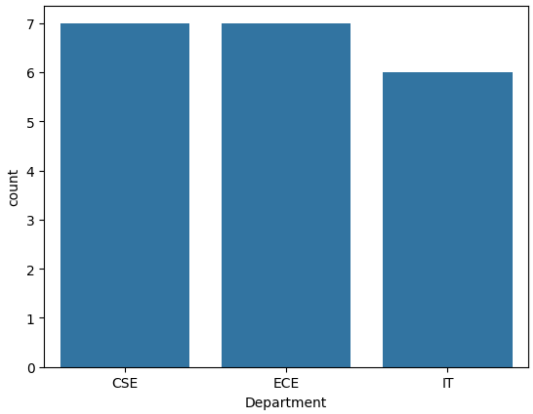

Study_Hours
Attendance
Marks
Assignment_Score

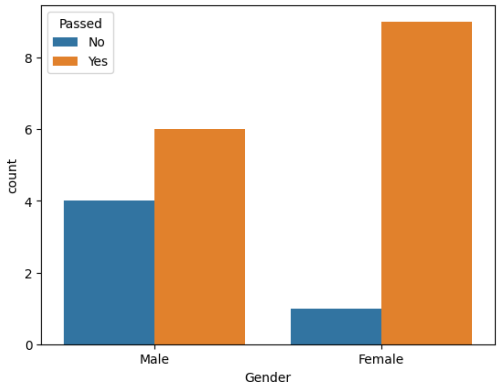

Study_Hours
Attendance
Marks
Assignment_Score
Study_Mode
Online
Offline

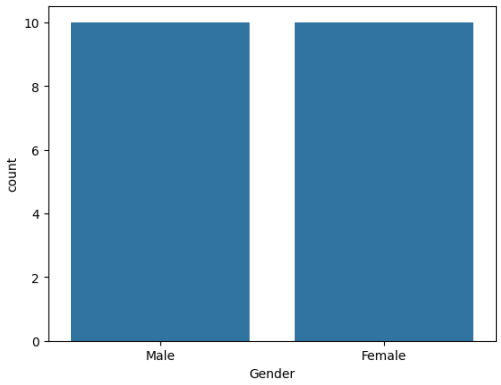

Study_Hours
Attendance
Marks
Assignment_Score
1
0.98
0.98
0.98
0.98
1
1
1
0.98
1
1
1
0.98
1
1
1
1.0000
0.9975
0.9950
0.9925
0.9900
0.9875
0.9850
0.9825
0.9800

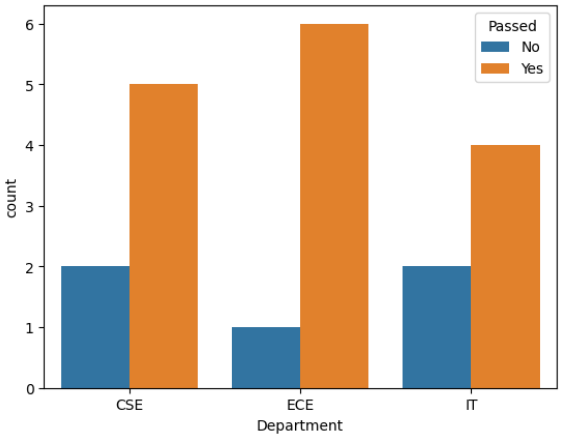

Marks
90
80
70
60
50

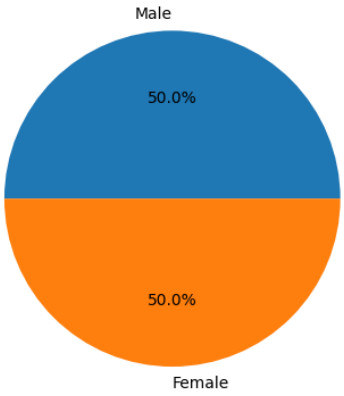

count
14
12
10
8
6
4
2
0
No
Yes
Passed


count
Department
CSE
ECE
IT
0
1
2
3
4
5
6
7


Passed
No
Yes
count
Gender
Male
Female
0
2
4
6
8


count
Gender
Male
Female
0
2
4
6
8
10


Passed
No
Yes
count
Department
CSE
ECE
IT
0
1
2
3
4
5
6


Male
50.0%
50.0%
Female

RESULT:

The given dataset was successfully loaded, explored, and visualised using Python, providing a better understanding of its data characteristics and relationships.

EXPERIMENT 17

AIM:

To implement a Linear Regression model for predicting marks based on study hours.

CODE:

```
import numpy as np
import pandas as pd

# Load the dataset
df = pd.read_csv('StudyHours_Marks.csv')
print(df)

# Display dataset information
df.info()

# Remove rows with missing values
df.dropna(inplace=True)

# Display updated dataset information
df.info()

# Display statistical summary
print(df.describe())
```

```
# Separate features and label
features = df.iloc[:, [0]].values
label = df.iloc[:, [1]].values

# Split the dataset into training and testing sets
from sklearn.model_selection import train_test_split

x_train, x_test, y_train, y_test = train_test_split(
    features, label, test_size=0.2, random_state=10
)

# Create and train the Linear Regression model
from sklearn.linear_model import LinearRegression

model = LinearRegression()
model.fit(x_train, y_train)

# Evaluate the model
print("Training Score:", model.score(x_train, y_train))
print("Testing Score:", model.score(x_test, y_test))

# Display coefficient and intercept
print("Coefficient:", model.coef_)
print("Intercept:", model.intercept_)
```

```
# Save the model
import pickle

pickle.dump(model, open('MarksPred.model', 'wb'))

# Load the saved model
model = pickle.load(open('MarksPred.model', 'rb'))

# Predict marks based on study hours
yr_of_exp = float(input("Enter Study Hours: "))

yr_of_exp_NP = np.array([[yr_of_exp]])
Salary = model.predict(yr_of_exp_NP)

# Display the predicted marks
print(
    "Estimated Marks for {} hours of study is {}".format(
        yr_of_exp, Salary
    )
)
```

OUTPUT:

1. Original Dataset

```
    StudyHours  Marks
0            1     35
1            2     40
2            3     45
3            4     50
4            5     55
5            6     60
6            7     65
7            8     70
8            9     74
9           10     79
10          11     83
11          12     87
12          13     91
13          14     94
14          15     97
```

2. Dataset Information

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 15 entries, 0 to 14
Data columns (total 2 columns):
 #   Column      Non-Null Count  Dtype
---  ------      --------------  -----
 0   StudyHours  15 non-null     int64
 1   Marks       15 non-null     int64
dtypes: int64(2)
memory usage: 372.0 bytes
```

3. Statistical Summary

```
       StudyHours      Marks
count   15.000000  15.000000
mean     8.000000  68.333333
std      4.472136  20.282528
min      1.000000  35.000000
25%      4.500000  52.500000
50%      8.000000  70.000000
75%     11.500000  85.000000
max     15.000000  97.000000
```

4. Linear Regression Model

```
LinearRegression()
```

5. Training Score

```
0.995281820510348
```

6. Testing Score

```
0.9961244386254701
```

7. Model Coefficient

```
array([[4.51970701]])
```

8. Model Intercept

```
array([32.0523195])
```

9. Final Prediction

```
Enter Study Hours: 4
Estimated Marks for 4.0 hours of study is [[50.13114754]]
```

RESULT:

The Linear Regression model was successfully created and used to predict marks based on the number of study hours.

EXPERIMENT 18

AIM:

To implement the K-Nearest Neighbours (KNN) classification algorithm to classify biscuits based on their weight and sweetness.

CODE:

```
import numpy as np
import pandas as pd

# Load the dataset
df = pd.read_csv('Fruit_KNN.csv')
df.info()

# Count the number of each fruit
print(df.Fruit.value_counts())

# Display the first five rows
print(df.head())

# Encode the Fruit column
from sklearn.preprocessing import LabelEncoder

encoder = LabelEncoder()
df['Fruit'] = encoder.fit_transform(df['Fruit'])
```

```
print(df.head())

# Separate features and label
features = df.iloc[:, :-1].values
label = df.iloc[:, 2].values

# Import required libraries
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier

# Split the dataset
xtrain, xtest, ytrain, ytest = train_test_split(
    features, label, test_size=0.2, random_state=42
)

# Create and train the KNN model
model_KNN = KNeighborsClassifier(n_neighbors=5)
model_KNN.fit(xtrain, ytrain)

# Evaluate the model
print("Training Accuracy:", model_KNN.score(xtrain, ytrain))
print("Testing Accuracy:", model_KNN.score(xtest, ytest))

# Generate the confusion matrix
from sklearn.metrics import confusion_matrix
```

```
print(confusion_matrix(label, model_KNN.predict(features)))


# Generate the classification report
from sklearn.metrics import classification_report


print(classification_report(
   label,
   model_KNN.predict(features)
))
```

OUTPUT:

1. Dataset Information

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 30 entries, 0 to 29
Data columns (total 3 columns):
 #   Column     Non-Null Count  Dtype
---  ------     --------------  -----
 0   Weight     30 non-null     int64
 1   Sweetness  30 non-null     float64
 2   Fruit      30 non-null     object
dtypes: float64(1), int64(1), object(1)
memory usage: 852.0+ bytes
```

2. Fruit Class Distribution

```
Fruit
Apple     10
Banana    10
Orange    10
Name: count, dtype: int64
```

3. Original Dataset – First Five Records

	Weight	Sweetness	Fruit
0	120	6.0	Apple
1	125	6.2	Apple
2	130	6.4	Apple
3	135	6.5	Apple
4	140	6.7	Apple

4. Label Encoding

	Weight	Sweetness	Fruit
0	120	6.0	0
1	125	6.2	0
2	130	6.4	0
3	135	6.5	0
4	140	6.7	0

5. KNN Model

```
KNeighborsClassifier()
```

6. Training and Testing Accuracy

```
Training Accuracy: 0.9166666666666666
Testing Accuracy: 0.6666666666666666
```

7. Confusion Matrix

```
array([[ 9,  0,  1],
       [ 3,  7,  0],
       [ 0,  0, 10]])
```

8. Classification Report

	precision	recall	f1-score	support
0	0.75	0.90	0.82	10
1	1.00	0.70	0.82	10
2	0.91	1.00	0.95	10
accuracy			0.87	30
macro avg	0.89	0.87	0.86	30
weighted avg	0.89	0.87	0.86	30

RESULT:

The KNN classification algorithm was successfully implemented, and biscuit classification performance was evaluated using accuracy, a confusion matrix, and a classification report.

EXPERIMENT 19

AIM:

To understand and implement the Logistic Regression algorithm for the given dataset.

CODE:

```
import numpy as np

import pandas as pd


# Load the dataset

df = pd.read_csv('Student_Pass_Logistic.csv')

print(df)


# Display the first five rows

print(df.head())


# Separate features and label

features = df.iloc[:, [0, 1]].values

label = df.iloc[:, 2].values


print(features)

print(label)


# Import required libraries

from sklearn.model_selection import train_test_split

from sklearn.linear_model import LogisticRegression
```

```
# Find suitable random states
for i in range(1, 401):
    x_train, x_test, y_train, y_test = train_test_split(
        features, label, test_size=0.2, random_state=i
    )

    model = LogisticRegression(max_iter=1000)
    model.fit(x_train, y_train)

    train_score = model.score(x_train, y_train)
    test_score = model.score(x_test, y_test)

    if test_score > train_score:
        print("Test {} Train {} Random State {}".format(
            test_score, train_score, i
        ))

# Split the dataset into training and testing sets
x_train, x_test, y_train, y_test = train_test_split(
    features, label, test_size=0.2, random_state=42
)

# Create and train the Logistic Regression model
finalModel = LogisticRegression(max_iter=1000)
```

```
finalModel.fit(x_train, y_train)

# Evaluate the model
print("Training Accuracy:", finalModel.score(x_train, y_train))
print("Testing Accuracy:", finalModel.score(x_test, y_test))

# Generate the classification report
from sklearn.metrics import classification_report

print(classification_report(
    label,
    finalModel.predict(features)
))
```

OUTPUT:

1. Original Dataset

	StudyHours	Attendance	Pass
0	1	50	0
1	2	55	0
2	2	60	0
3	3	58	0
4	3	62	0
5	3	65	0
6	4	60	0
7	4	68	1
8	4	70	1
9	5	65	0
10	5	70	1
11	5	72	1
12	6	68	1
13	6	75	1
14	6	78	1
15	7	72	1
16	7	80	1
17	7	82	1
18	8	78	1
19	8	84	1
20	8	86	1
21	9	82	1
22	9	88	1
23	9	90	1
24	10	86	1
25	10	92	1
26	10	94	1
27	11	90	1
28	11	95	1
29	12	96	1

2. Features

```
array([[ 1, 50],
       [ 2, 55],
       [ 2, 60],
       [ 3, 58],
       [ 3, 62],
       [ 3, 65],
       [ 4, 60],
       [ 4, 68],
       [ 4, 70],
       [ 5, 65],
       [ 5, 70],
       [ 5, 72],
       [ 6, 68],
       [ 6, 75],
       [ 6, 78],
       [ 7, 72],
       [ 7, 80],
       [ 7, 82],
       [ 8, 78],
       [ 8, 84],
       [ 8, 86],
       [ 9, 82],
       [ 9, 88],
       [ 9, 90],
       [10, 86],
       [10, 92],
       [10, 94],
       [11, 90],
       [11, 95],
       [12, 96]])
```

3. Labels

```
array([0, 0, 0, 0, 0, 0, 0, 1, 1, 0, 1, 1, 1, 1, 1, 1, 1, 1,
       1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1])
```

4. Final Logistic Regression Model

```
LogisticRegression(max_iter=1000)
```

5. Training and Testing Accuracy

```
Training Accuracy: 1.0
Testing Accuracy: 1.0
```

6. Classification Report

	precision	recall	f1-score	support
0	1.00	1.00	1.00	8
1	1.00	1.00	1.00	22
accuracy			1.00	30
macro avg	1.00	1.00	1.00	30
weighted avg	1.00	1.00	1.00	30

RESULT:

The Logistic Regression algorithm was successfully implemented and used to classify the given dataset.

EXPERIMENT 20

AIM:

To implement the K-Means clustering algorithm on a given dataset.

CODE:

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

# Load the dataset
df = pd.read_csv('Cafe_Customers_KMeans.csv')

# Display dataset information
df.info()

# Display first five rows
print(df.head())

# Visualize the dataset
sns.pairplot(df)
plt.show()

# Select features for clustering
features = df.iloc[:, [1, 2]].values
```

```
# Apply K-Means clustering
from sklearn.cluster import KMeans

model = KMeans(n_clusters=3, random_state=42, n_init=10)
model.fit(features)

# Create final DataFrame with cluster labels
Final = df.iloc[:, [1, 2]].copy()
Final['label'] = model.predict(features)

print(Final.head())

# Visualize the clusters
sns.set_style("whitegrid")

sns.FacetGrid(Final, hue='label', height=6) \
    .map(plt.scatter, 'MonthlyVisits', 'AverageSpend') \
    .add_legend()

plt.show()

# Elbow method to find the optimal number of clusters
features_el = df.iloc[:, [1, 2]].values
```

```
wcss = []

for i in range(1, 10):
    model = KMeans(
        n_clusters=i,
        random_state=42,
        n_init=10
    )
    model.fit(features_el)
    wcss.append(model.inertia_)

# Plot the Elbow graph
plt.plot(range(1, 10), wcss, marker='o')
plt.xlabel('Number of Clusters')
plt.ylabel('WCSS')
plt.title('Elbow Method')
plt.show()

# Final K-Means model
finalModel = KMeans(
    n_clusters=3,
    random_state=42,
    n_init=10
)
```

```
finalModel.fit(features)


# Assign final cluster labels
Final['label'] = finalModel.predict(features)


print(Final)
```

OUTPUT:

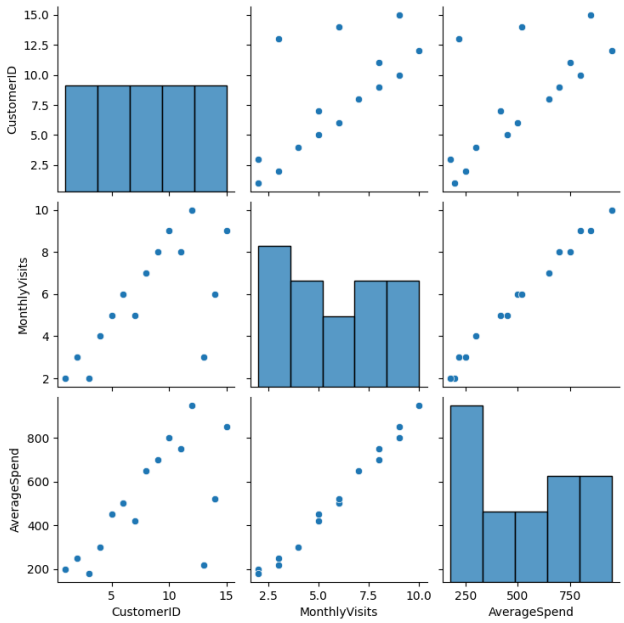
1. Dataset Information

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 15 entries, 0 to 14
Data columns (total 3 columns):
 #   Column         Non-Null Count  Dtype
---  ------         --------------  -----
 0   CustomerID     15 non-null     int64
 1   MonthlyVisits  15 non-null     int64
 2   AverageSpend   15 non-null     int64
dtypes: int64(3)
memory usage: 492.0 bytes
```

2. Original Dataset – First Five Records

	CustomerID	MonthlyVisits	AverageSpend
0	1	2	200
1	2	3	250
2	3	2	180
3	4	4	300
4	5	5	450

3. Pair Plot



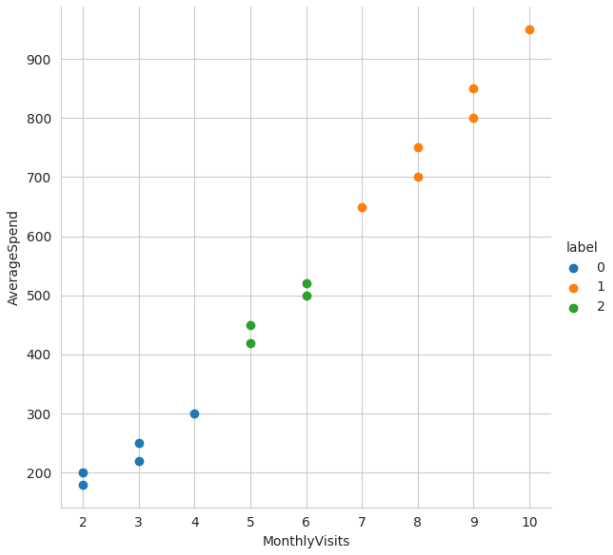
4. K-Means Model

```
KMeans(n_clusters=3, n_init=10, random_state=42)
```

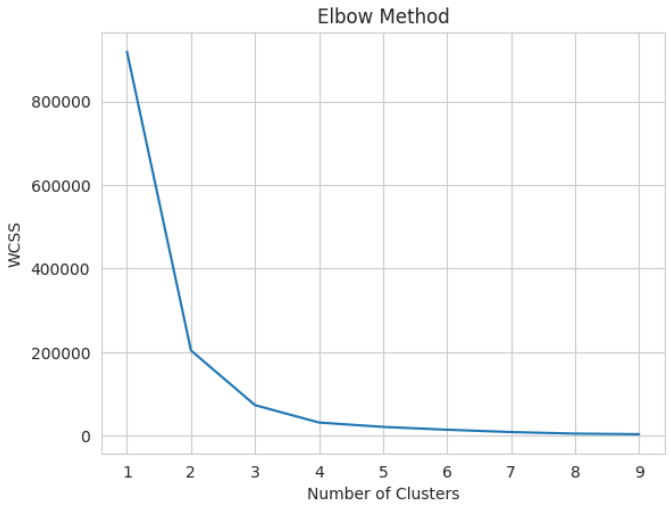
5. Clustered Data – First Five Records

	MonthlyVisits	AverageSpend	label
0	2	200	0
1	3	250	0
2	2	180	0
3	4	300	0
4	5	450	2

6. Cluster Visualization



7. Elbow Method – WCSS



8. Final K-Means Model

```
KMeans(n_clusters=3, n_init=10, random_state=42)
```

9. Final Clustered Dataset

	MonthlyVisits	AverageSpend	label
0	2	200	0
1	3	250	0
2	2	180	0
3	4	300	0
4	5	450	2
5	6	500	2
6	5	420	2
7	7	650	1
8	8	700	1
9	9	800	1
10	8	750	1
11	10	950	1
12	3	220	0
13	6	520	2
14	9	850	1

10. Final Cluster Output

```
Customer groups are represented by the labels 0, 1 and 2.
The final dataset shows the cluster assigned to each customer
based on
MonthlyVisits and AverageSpend.
```

RESULT:

The café customers were successfully grouped into clusters using K-Means clustering based on their monthly visits and average spending.